

BachelorThesis Application Note

Elektronics-ICT 2025-2026

UAV with object avoidance project

<i>Author</i>	<i>Samy Warnants</i>
<i>PXL coach</i>	<i>Stijn Boutsen</i>
<i>Teamlead</i>	<i>Tommi Kokko</i>
<i>Company</i>	<i>Lapland University of Applied Sciences</i>

Abstract

This application note investigates the design and development of a modular UAV platform capable of autonomous obstacle avoidance while operating reliably in harsh Finnish winter conditions. The study explores both the technical feasibility and practical limitations of combining environmental robustness with intelligent navigation in a single drone system. The increasing demand for search-and-rescue support in extreme climates highlights the need for UAVs that can function in low temperatures, poor visibility, and unpredictable terrain. This research addresses the question of whether a modular drone platform can be developed to perform obstacle avoidance effectively under these conditions, while remaining versatile enough to serve as a broader educational and research platform.

To support component selection and system design, tools such as Ecalc and the Ardupilot documentation were used extensively. These resources enabled informed decisions regarding propulsion systems, power requirements, and flight controller configuration, ensuring a technically sound foundation for the platform. The results demonstrate that, while a functional prototype can be realized, several integration challenges emerged during development, particularly related to system stability and environmental constraints. Although the core research question proves technically feasible, the available development time was insufficient to fully resolve all encountered issues. Nonetheless, the implemented system establishes a solid baseline, confirming the viability of the concept while exposing critical areas for refinement.

In conclusion, the project validates that a modular, obstacle-avoiding UAV for cold environments is achievable, but requires extended development and iterative testing to reach full operational reliability. The work carried out forms a strong foundation for future development. Looking forward, this platform offers significant potential for further optimization, including improved sensor integration, enhanced cold-weather resilience, and expanded autonomy features, making it a valuable stepping stone toward robust UAV deployment in Arctic search-and-rescue scenarios and beyond.

Table of contents

1	Introduction.....	4
1.1	Problem case and situation.....	4
1.2	Thesis question	5
1.3	Lookahead and what to expect as a result	6
2	Materials and methods	7
2.1	Autonomous flight components	7
2.1.1	Object avoidance methods.....	7
2.1.2	Combination	10
2.1.3	Jetson Orin	11
2.2	Drone.....	12
2.2.1	Part selection	12
2.2.2	Flight controller	13
2.2.3	Flight controller firmware.....	14
2.2.4	Motors, ESC's and propellers.....	15
2.2.5	Transmitter and receiver	16
2.2.6	Frames	17
2.2.7	Battery	20
2.2.8	How to prepare the drone for harsh environments?	21
2.2.9	Obstacle avoidance logic	22
2.3	Software setup and hardware connections.....	24
2.3.1	Flight controller first time setup.....	24
2.3.2	Sensor calibration	25
2.3.3	Transmitter and receiver setup	27
2.3.4	Motor and ESC setup and calibration	30
2.3.5	Jetson setup and flight controller communication.....	31
2.3.6	Final schematics.....	34
2.4	Testplan and flighttesting	36
2.4.1	Phase 1 full system test	36
2.4.2	Phase 2 simple outdoor propeller test	36
2.4.3	Phase 3 obstacle avoidance test.....	37
3	Fase 3: Results	38
3.1	Drone frame and setup	38
3.2	Autonomous flight code.....	39
4	Discussion	40
4.1	Technical reflection on the results.....	40
4.2	SDG-reflection	42
4.3	Value for the workplace.....	43

4.4	Reflection for the X-factor	44
5	Conclusion	45
6	Bibliografy.....	46
	Code prototype 1 flight controller communication:	50
	Code prototype 2 depth camera:	54

1 Introduction

1.1 Problem case and situation

Originally founded in 1999 was PLab (P standing for Game and Lab short for laboratory of course) and it's goal originally was to develop virtual reality games with financial aid from the ERDF (European Regional Development Fund). At first this was a simple team of 10 engineers who always focused on utilizing the newest technologies in their virtual reality projects (Lapin AMK, sd). With time this goal expanded as to where the team is now consisting of 30 members and the focus has been to work and cooperate with both national and international partners some of these partners are: Lapland University of Applied Sciences, the Faculty of Arts of the University of Lapland and the Lapland Education Centre REDU nationally as well as internationally teams such as PXL. The lab allows students in ICT to experience the internship experience with real life projects. Under the Frostbit team there are several separate individual teams most relevant to this project is the ARC team lead by team leader Tommi Kokko above him is Tauno Tepsa. In this team everyone is focused on hardware and software solutions to problems faced by companies or partners. This new lab is referred to as the Frostbit lab.



Figure 1: Frostbit lab logo

1.2 Thesis question

One of the projects they require assistance with is the creation of an UAV (unmanned aerial vehicle) that can autonomously avoid objects during flight. This UAV will also serve as research and learning platform for future students to either improve on what has been created, create payloads for expanding its functionality. The project goal can be described like so: make it so that it can fly in harsh environments and do both object avoidance and people rescue keeping headroom for future improvements. Relevance of the project comes from the fact that this is something that can be useful for them for possible emergency services or for students in the future who will utilize the platform to learn how to work with drones. One other example is that sometimes the connections here have trouble staying connected for 5g or GPS. What a drone could do is act as a flying repeater, a team of drones could provide full coverage where GPS or satellites cannot otherwise reach (Lapin AMK, 2025). In the image below is an example of how that could work from the frostbit website.

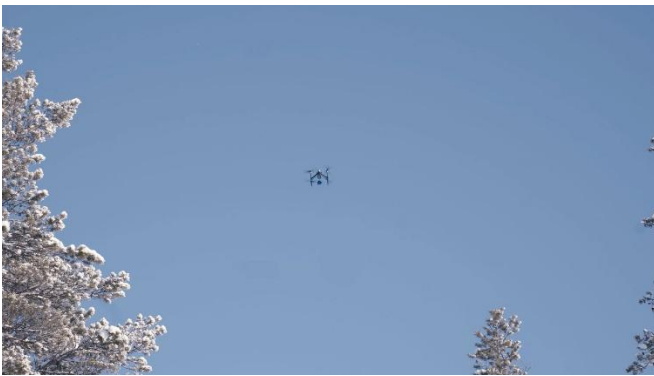


Figure 2: Frostbit drone flying with antenna

This is also a good example of how an object avoiding drone could be of great use since it will need to maintain altitude for a reasonable amount of time. But also, because this is an example of a module that could be made to work with the drone. If the drone is made to be a movable antenna the object avoidance comes to play because it will have to manoeuvre around obstacles to provide an optimal connection to the users making use of the connection (image below for visual reference). Or more could be built to provide a mesh network for optimal connectivity. So, the main relevance of the project is to provide the groundwork for the project so that it can be worked on more later.

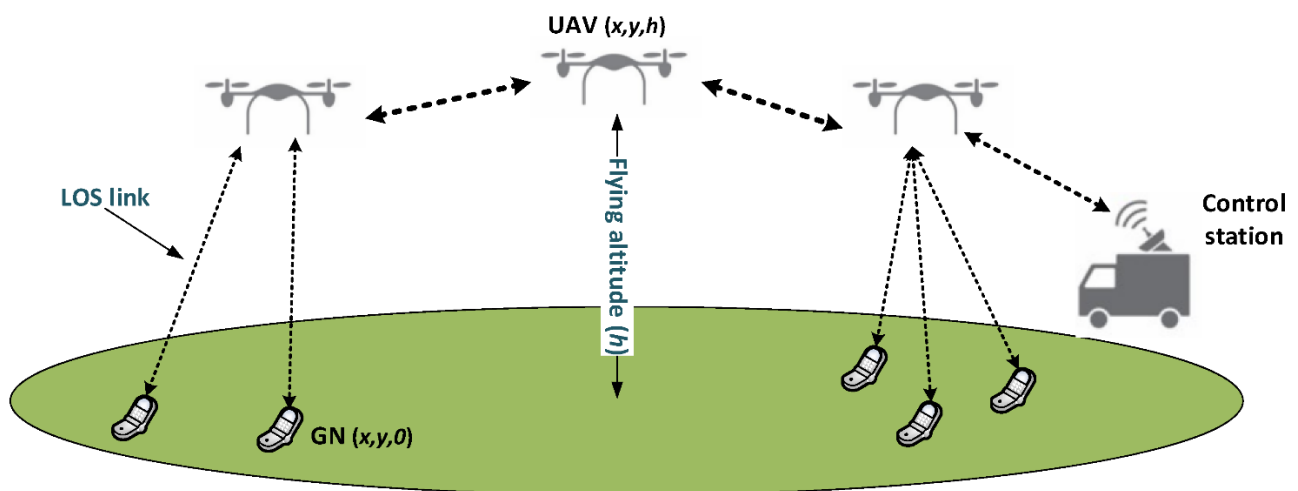


Figure 3: Example of a multidrone networking concept

1.3 Lookahead and what to expect as a result

This then decides as to what the goal is for this project. If it was not clear by now is simply this: “Create a UAV that can do object avoidance powered with artificial intelligence and document the testing steps and creation”. With some things to keep in mind: There needs to be an option available for future payloads to be included on board and if possible, figure out how to add lidar for better object avoidance. Making sure that costs are not too high. What they expect for me is a drone with TRL (Technology Readiness Level image reference for reference) of 3-4, which pretty much means that it is expected that a concept of the idea is worked out and the basics are there for future students to improve on and work on further.

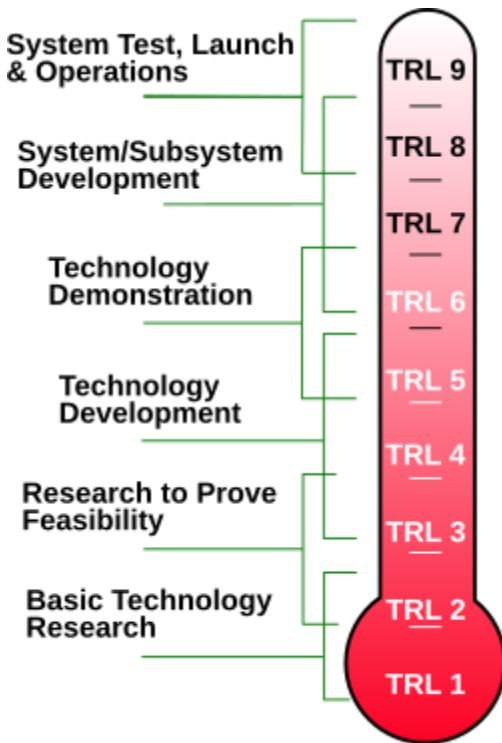


Figure 4: TRL reference guide

2 Materials and methods

2.1 Autonomous flight components

2.1.1 Object avoidance methods

2.1.1.1 Lidar

LiDAR (Light Detection and Ranging) has become an essential sensing technology in modern unmanned aerial vehicle (UAV) systems, particularly for applications requiring precise distance measurement and environmental awareness. The primary advantage of LiDAR lies in its ability to deliver accurate and high-frequency distance measurements independent of lighting conditions. This makes it especially suitable for tasks such as altitude estimation, terrain following, and obstacle avoidance. When combined with Ardupilot, LiDAR data can be directly utilized through built-in rangefinder and proximity libraries, enabling real-time control adjustments without the need for complex external processing. This tight integration reduces system latency and enhances overall flight stability, particularly in low-altitude or GPS-denied environments. Ardupilot is a favourable choice for LiDAR integration because of its modular design and support for multiple communication protocols, including UART, I2C, and CAN. This flexibility allows a wide range of LiDAR sensors to be deployed depending on system requirements. Furthermore, Ardupilot provides configurable parameters and logging capabilities, which facilitate sensor calibration and performance evaluation key aspects in a research or thesis context. (Ardupilot, sd)



Several LiDAR options can be considered when working with Ardupilot, each presenting distinct advantages

Figure 5: Example of a compatible lidar

and limitations. Low-cost sensors such as the TFmini series are attractive for their simplicity, low weight, and ease of integration, making them suitable for basic altitude hold applications. However, their limited range and sensitivity to environmental conditions restrict their use in more demanding scenarios. Mid-range solutions, such as the Garmin LIDAR-Lite, offer improved range and performance at a moderate cost, though they may require additional filtering to mitigate noise. High-end sensors, including those from Light Ware, provide superior accuracy and extended range, making them well-suited for professional or research-grade applications, albeit at a significantly higher cost and power consumption. For more advanced use cases, such as full obstacle detection and mapping, scanning LiDAR systems (e.g., 2D rotating LiDAR's) or solid-state LiDAR technologies can be employed. These sensors provide richer spatial information but introduce additional complexity, often necessitating the use of a companion computer for data processing. This increases system weight, power requirements, and integration effort, which must be considered during system design. In summary, LiDAR combined with Ardupilot offers a robust and scalable solution for UAV sensing and navigation. However there remain other options to be considered.

2.1.1.2 Depth camera's

Depth cameras have become increasingly important in autonomous UAV systems due to their ability to provide real time three-dimensional environmental perception. Unlike traditional monocular cameras, depth cameras can estimate the distance between the sensor and surrounding objects. This enables applications such as obstacle avoidance, simultaneous localization and mapping (SLAM) (IEEE, 2016), and autonomous navigation. In this project the selected depth sensing solution was the Intel RealSense D415 depth camera. Although alternative systems such as the Stereolab's ZED 2 were also considered. The Intel RealSense D415 is a stereo-based RGB-D camera that combines two infrared sensors with an RGB camera to generate depth information using active stereoscopic vision. The camera provides depth resolutions up to 1280×720 pixels and supports high frame rates, making it suitable for real-time robotic applications. Another one of the primary reasons for selecting the D415 was its balance between performance, size, cost, and ease of integration. Its compact form factor and relatively low power consumption make it well-suited for UAV platforms where payload weight and onboard energy availability are limited. Another significant advantage of the D415 is its strong software ecosystem. The camera is supported through the Intel RealSense SDK and is compatible with middleware frameworks such as ROS (Robot Operating System), which simplifies integration with Ardupilot companion computer architectures. Furthermore, the D415 offers accurate short- to medium-range depth perception, making it particularly effective for indoor navigation and close-range obstacle detection. Studies evaluating the D415 have also demonstrated stable and accurate pose estimation performance for robotics and tracking applications. (Intel, sd)



Figure 6: Intel D415 reference image

Despite these advantages, the D415 also presents several limitations. The camera relies on active infrared projection, which can reduce performance in environments with strong sunlight or highly reflective surfaces. In addition, the rolling shutter design may introduce motion artifacts during rapid UAV movement. Its effective depth range is also more limited compared to higher-end stereo systems, particularly in outdoor environments. An alternative considered during system selection was the Stereolab's ZED 2 depth camera. The ZED 2 is a more advanced stereo vision system capable of long-range depth sensing, wide field-of-view imaging, and integrated spatial tracking through onboard inertial sensors. The system provides accurate depth measurements up to approximately 20 meters and includes additional sensors such as an IMU, magnetometer, and barometer. These features make the ZED 2 particularly suitable for large-scale outdoor navigation and advanced autonomous operations. The ZED 2 also offers advantages in terms of environmental perception. Its wider field of view and enhanced low-light performance improve situational awareness and mapping capabilities. Additionally, the camera is optimized for AI-driven applications such as object detection and SLAM. However, these capabilities come at the cost of increased computational requirements. The ZED 2 typically requires a dedicated GPU-enabled companion computer for real-time depth processing, increasing system complexity, power consumption, and overall payload weight. Furthermore, the camera is significantly more expensive than the D415, which may not be justified for applications focused primarily on short-range obstacle avoidance.



Figure 7: ZED 2 reference image

In summary, both the Intel RealSense D415 and the Stereolab's ZED 2 provide valuable depth sensing capabilities for UAV applications. The ZED 2 offers superior range, field of view, and environmental perception, making it highly suitable for advanced autonomous systems. However, the Intel RealSense D415 was selected for this project due to its lower cost, compact design, ease of integration, and sufficient performance for the intended application. Its compatibility with existing robotic software frameworks and lower hardware requirements made it a practical and efficient choice for integration into the UAV platform.

2.1.2 Combination

The combination of LiDAR and depth cameras represents a sensing strategy in UAV systems, leveraging the strengths of both technologies to overcome their individual limitations. While LiDAR provides highly accurate and reliable distance measurements along specific directions, depth cameras offer dense spatial information across a wider field of view. Integrating both sensors to operate together on one framework enables stronger perception and improves overall system reliability, particularly in difficult environments. One of the primary advantages of combining LiDAR with a depth camera such as the Intel RealSense D415 is the enhancement of depth perception accuracy and redundancy. LiDAR sensors are generally more reliable for precise distance estimation, especially in scenarios where exact altitude or ground clearance must be maintained. In contrast, the D415 provides a dense depth map, allowing the system to detect and interpret the shape and structure of nearby obstacles. By fusing these data sources, the UAV can benefit from both high-precision point measurements and environmental context. This combination is particularly good in environments where one sensing technology works not that well. For instance, depth cameras relying on infrared projection, such as the D415, can experience reduced performance under strong sunlight or in featureless environments. In such cases, LiDAR can provide stable and accurate measurements unaffected by lighting conditions. Conversely, LiDAR sensors typically offer limited spatial coverage (especially single-point variants), making it difficult to detect obstacles outside their narrow sensing direction. The depth camera compensates for this limitation by providing a broader field of view and enabling obstacle detection across a larger portion of the scene.

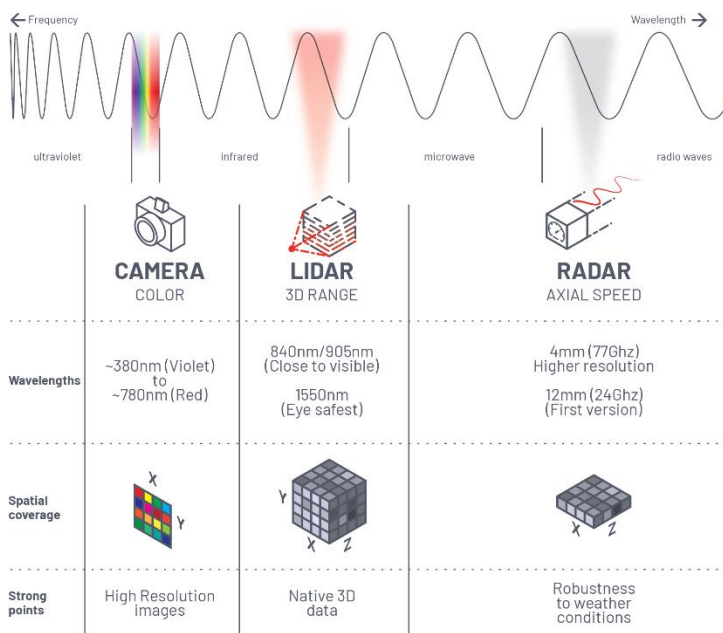


Figure 8: Wavelength comparison (Roe, 2023)

From a system design perspective, sensor fusion can also improve robustness and fault tolerance. By cross-validating measurements from LiDAR and the depth camera, erroneous readings can be identified and filtered. This is particularly relevant in safety-critical UAV applications, where reliance on a single sensor may introduce risk. Additionally, combining both sensors enables more reliable operation in diverse environments, including indoor, outdoor, and GPS-denied scenarios. However, the integration of multiple sensors introduces additional complexity. Data from LiDAR and depth cameras must be synchronized and processed, often requiring a companion computer capable of handling increased computational loads. Furthermore, sensor calibration and alignment are necessary to ensure that measurements from both sources are spatially consistent. These requirements increase system design effort and may impact power consumption and payload constraints. Therefore it may be best to stay with depth camera sensing.

2.1.3 Jetson Orin

The NVIDIA Jetson Orin is a compact edge AI platform from NVIDIA that integrates a powerful GPU, ARM CPU, and dedicated AI accelerators, enabling real-time processing of complex workloads such as computer vision and sensor fusion directly onboard. Compared to the older NVIDIA Jetson Nano it offers dramatically higher compute performance (tens to hundreds of TOPS versus ~ 0.5 TOPS). This allows it to run advanced neural networks, handle multiple camera streams and perform depth processing without becoming a bottleneck. Although this comes at the cost of higher power consumption, greater heat output, and increased system complexity. In an AI drone setup, especially when combined with a depth camera (e.g., stereo or LiDAR-based) and a flight controller like the Cube Orange the Jetson Orin acts as a companion computer responsible for high-level perception and decision-making such as: obstacle detection, path planning, and environment mapping, while the flight controller handles low-level stabilization and real-time motor control. This separation is critical because it ensures deterministic flight behaviour while still enabling advanced autonomy. The Orin's processing power makes it particularly well-suited for fusing depth data with vision algorithms to achieve reliable obstacle avoidance in real time, even in challenging conditions, and its compatibility with frameworks like ROS and MAVLink allows seamless communication with the flight controller, making it an ideal choice for a modular, scalable AI-driven drone platform.

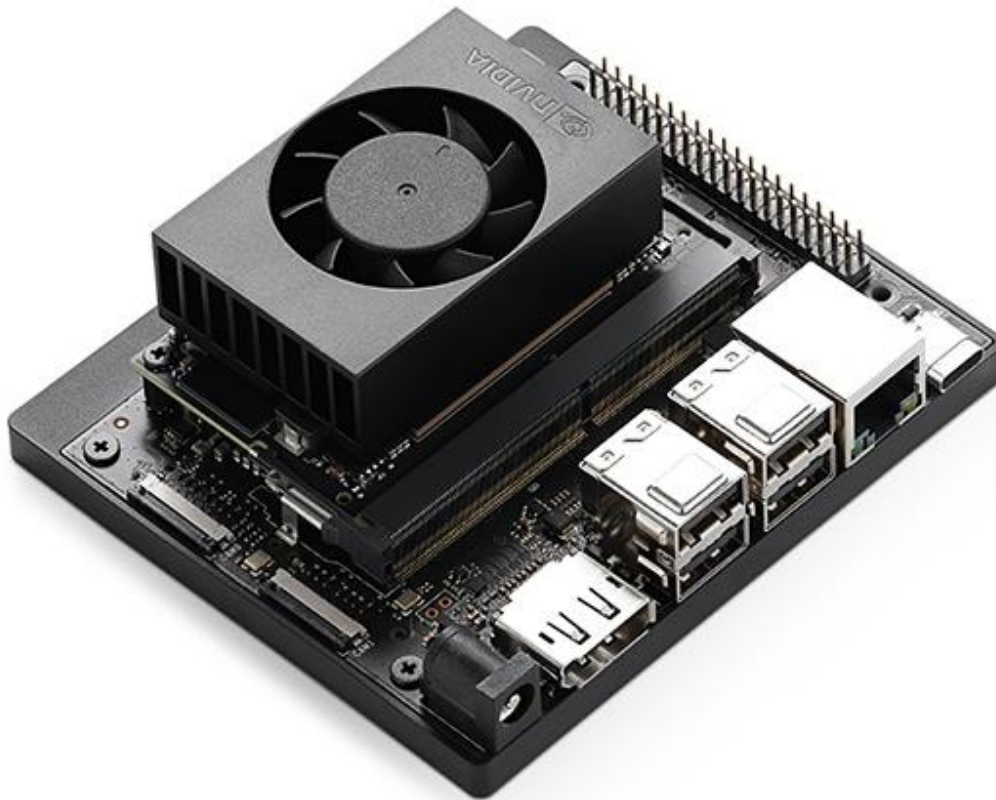


Figure 9: Jetson orin nano reference image

2.2 Drone

2.2.1 Part selection

For this project a few different sites and programs were needed. The most important one to mention is Ecalc it is a general tool that can be used to calculate drone efficiency based upon user given variables. This can be weight, maximum height, frame size, propeller length and pitch, battery capacity and so much more. There are two versions of the tool the one used to determine the component specifications is the easy version there is a more complex version, but this one is way more complex than what is needed for this project. The most important factor in drone building is the drone weight. This is because this determines the speed license class and the flight time among other factors. So, to determine the minimum weight, it's best to see what will be needed on the drone starting with the weight of the most important parts. This is the flight controller and the companion computer. Combining those two weights and then doubling them gives us a rough estimate of the weight that the drone will need to carry.



Figure 10: Ecalc reference image for the correct tool

2.2.2 Flight controller

Starting with the flight controller there is a quite a lot of good options in the market. But to make the right choice it must be understood what the purpose is of the flight controller on the drone. A flight controller is there for when you wish to simplify the control, connections and setup of the drone and indirectly make the wiring of the drone a lot less complicated. The flight controller will handle everything that the drone needs from battery management to receiver communication to ensure a safe and reliable flight. Setup for them is simple and most of it is well documented online outside of course the usual smaller, rarer issues that often cannot be explained. The one already available in the lab is the Pixhawk c6 even though for its price it's a great option it has one small problem. Future upgradability is not that great on it as well. From the Pixhawk family there is one that does offer this but at a high cost it is the Pixhawk 6x CUAV version. Alternatively, there is the cube pilot family the main one of interest being the orange cube+, it comes in with a price point that is fairly near the Pixhawk systems, but it does offer more advantages over the Pixhawk c6. The main part being its operating temperatures they lay between -10°C and 55°C, it also offers triple redundant imu's something that is lacking on the Pixhawk systems along with internal vibration isolation systems to make the gyro that much more accurate. It has some downsides as well however since it is not waterproof it may bring in troubles when condensation comes to play. It does also save on space as motor connectors are directly integrated onto the same board. This makes wiring a lot less complicated and more straight forward.



Figure 11: Reference image of a regular Orange Cube

2.2.3 Flight controller firmware

Two of the most widely used open-source flight control platforms for unmanned aerial vehicles are PX4 and Ardupilot. Both platforms provide core flight control functionality, autonomous mission execution, sensor integration and support for companion computers. Despite these similarities, several differences exist in their architecture, feature sets and intended applications. PX4 is designed with a modern and modular software architecture that emphasizes flexibility and integration with robotics frameworks such as ROS 2. The platform is widely used in academic research and experimental UAV applications due to its clean codebase and developer-oriented design. PX4 supports autonomous navigation, waypoint missions, and companion computer integration, making it suitable for advanced research projects involving artificial intelligence and robotics. Ardupilot on the other hand, is one of the most mature and extensively tested autopilot platforms available. It supports a broader range of vehicle types, including multicopters, fixed-wing aircraft, helicopters, rovers, boats, and submarines. Compared to PX4, Ardupilot offers a larger set of built-in flight modes, autonomous behaviours, failsafe mechanisms, and mission-planning capabilities. Furthermore, it benefits from a large international user community, extensive documentation, and a significant collection of tutorials and troubleshooting resources.



Figure 12: PX4 versus AruPilot

For this project, Ardupilot was selected as the preferred flight control software. The primary reason for this decision is its proven reliability and comprehensive feature set. Since the UAV platform combines a Cube Orange+ flight controller with an NVIDIA Jetson Orin Nano companion computer and an Intel RealSense D415 depth camera, stable communication and extensive support for companion computer integration are essential. Ardupilot provides mature MAVLink functionality, robust obstacle avoidance support, and well-documented interfaces for external computers performing vision-based processing. Additionally, the large user community and extensive documentation significantly reduce development risks and troubleshooting time. This is particularly important within the limited timeframe of the project, as it allows more effort to be focused on the development of the obstacle avoidance system rather than resolving flight control software issues. While PX4 remains an excellent platform for research and future development, Ardupilot provides a more complete, stable, and practical solution for implementing an autonomous UAV capable of reliable operation in real-world environments. Therefore, Ardupilot was considered the most suitable choice for achieving the objectives of this project.

2.2.4 Motors, ESC's and propellers

For the motors and ESC's, the ones that came with the drone test frame are the BLheli 30A combined with generic 2212 KV850-920 motors. There is no prior documentation available on why it was chosen for this frame but researching what is compatible with the s500 v2 the current reasoning appears to be because it is within recommended specifications for this drone frame. As for the new frame the decision was made to research how the default kit is built and then expand upon those specifications to make it for the project. This resulted in utilizing the recommended combination of the T-motor MN4014 with a kv of 400 with the generic T-motor 40A ESC as it comes recommended for this motor and spec on the datasheet. (It should be noted that there is also a 300 kv version but since it has the same weight and similar power usage as the 400 kv version it resulted into that being the final choice). Before this there were the SunnySky USA motors specifically V4014 400kv for the new frame and the SunnySky X2814 v3's. These were also a good alternative for the frames, but tariffs and long delivery time required the pivot to the T-motor combination. For the test frame the propellers that were used were DJI 9-inch 4.5 pitch propellers. For the new frame there were two options to choose from for the propellers. One of them is T-Motor P15x5 it was priced, light weight and offered high strength and stability, however this is a now out of stock option so instead the NS15x5 was chosen as they offer the same features but at a slightly higher price than the P15x5.



Figure 13: Reference image for MN4014 motor kit

2.2.5 Transmitter and receiver

The transmitter and receiver required less thought as there are a few protocols that can be used for RC communication. This is a list of protocols that are automatically connected to the Ardupilot software as an example: PPM, SBus, FPort, Crossfire and ELRS, DSM, DSM2, DSM-X Satellite receivers, IBUS, MULTIPLEX SRXL. Each has their own advantages and disadvantages. The test frame had a Skydroid R10 transmitter receiver combo which uses FHSS. But this had to be changed since there is something that is significantly more modern than FHSS which is ELRS. This is what is typically used inside the RC world for its ease of setup and low power usage. Wiring it to the flight controller is also a breeze to be done. (given of course that both the transmitter and receiver modules both are on a correct version). For the new frame the decision was made to utilize the Radiomaster TX16S since this is already present in the lab and it offers a lot of setup options for the user regarding switches and the ability to switch out backpacks for possible newer communication types in the future. For the receiver the decision was made to utilize the 4TD ExpressLRS 2.4GHz True Diversity Receiver, its setup for both is incredibly straightforward and well documented on the Radiomaster website. But it is important to choose the LBT version not the FCC version for use with the Europa due to the litigations regarding signal strengths.



Figure 14: Reference image TX16s transmitter from Radiomaster

2.2.6 Frames

2.2.6.1 Frame 1

There were a few frame idea's that were worked out were based upon what was available in the lab this makes the project more durable as older components that are still functional get used again and there is no need to wait for parts to be delivered. The first of these was to simply make a 3D printed frame myself from scratch, this would offer the option of immediately having the frame available and having full choice over the components, but it does bring the downside that a lot of time will have to be spent in creating a good strong and compatible frame. Not to mention that it could be costly as stronger 3D filaments that fit the strength needs have a higher price tag. Since having a frame is not the goal of the project this option was considered but not chosen. After that a similar but close idea was to modify the s500 v2 frame with 3d prints to make it bigger, but it would run into the same problems as a fully 3D printed frame with the added difficulty that it will need to fit existing parts requiring more prototyping and thus more filament usage increasing the cost. Therefor the idea to 3D print for this project was limited to the parts that are purely required for drone operationality. The problem with this kind of frame would also be that some filaments have troubles surviving against the though weather or moisture issues.



Figure 15: Photo from testflight with the testframe

2.2.6.2 Frame 2

Going back to see the provided options from the lab, there is another complete frame that is available inside of the lab the GeoDrone X4L V4.1.4. This is a more commercial option that originally came with a thermal camera addon. It is a big frame with a carrying case that is strong and its size allows for more parts to be fit inside of the main part of the drone. The problem it has is that it had suffered a crash and therefore one of its propellers was broken. When the research was done it did not seem like there was a cost-effective propeller replacement available online making one from 3D printing would have been an option, but it is uncertain how the stability would change not to mention flight time and the risk of it breaking again. Another risk is the unknown status of how much and what is still operational for this specific frame. With all these risks kept in mind as well as keeping track of project time to achieve the thesis goal it was best to move on and find an alternative to this frame.



Figure 16: Reference image for the GeoDrone

2.2.6.3 Frame 3

The one recommend to me by my project manager was an Iflight drone frame, this would be an easy to assemble and easily available option to use since most parts for it were already present in the lab. The downside of it however is that it is a small frame and mounting any Jetson would consume too much power as well as weigh too much for the small frame to carry. Not to mention that it would leave no room for future expansion or modularity which at the time was a consideration.



Figure 17: Iflight frame

2.2.6.4 Frame 4

The only frames that are left to discuss are the ones used for this project. The Holybro s500 v2 this frame though small was big enough for this project lacking in only one place. Limited space for place the Jetson and the components needed, as well as the previous documentation on this specific frame and it's components being lacking making it hard to become the certain choice to go with for this project. The final reason as to why this frame is not suitable for the project is that the recommended propellers are no longer in stock from a reputable source making it's durability completely dependent on the success of future flights.



Figure 18: Image from before modifying and testing the test frame

2.2.6.5 Frame 5

Finally, there is the x650 frame from Holybro, it has all the advantages of the s500 vs the benefit of allowing a larger cargo easier transport since the arms can be folded collapsed making it's easier to carry. Otherwise, it is a more recent frame that allows lower kv motors to be used which are more efficient with bigger propellers making up from the lost trust from the reduced motor power. It also has a reserved space for the flight controller and the companion computer to be mounted and since it is more modular it is even possible in the future to make the addition of lidar to the frame of the drone by creating a compatible 3D mount for it. And since it is more recent then the s500 v2 support for it will be easier to find as well as replacement components as the only part that was not able to be found from the parts list of the original drone frame is the ESC's as they are no longer for sale from an EU reseller. It also has foldable arms for ease of transport and better cable management for the drone.



Figure 19: Reference image of the empty final frame

2.2.7 Battery

For the battery the biggest challenge was trying to figure out a way to make it work in the extreme Finnish colds. One way to solve this problem is to find a battery that is rated to survive these colder temperatures but those are expensive not to mention hard to find. Then there is also the way of figuring out a heating system to keep the batteries at operating temperature during flight. The problem with this is that heating up batteries is not safe to do mid-flight and reduces efficiency as well as being a fire risk. Therefore the most logical option that makes sense is to prepare a reserve battery and keep it in a warm environment during testing and switch out old cold ones with new warmer ones after long flights. From my research a temperature between 20 and 25°C should be sufficient (Hobbyking, sd). Having solved this problem the next decision was which battery should be used. Using Ecalc this can be easily determined inputting all the required environmental information and matching the chosen parts looking up each weight combining them and inputting multiple battery options. The result is a 10.000 MAH 6s2P battery as this offers the right voltage for all components, it brings in enough power long term to compensate for its heavier weight but brings in the downside of requiring a large amount of charging time.



Figure 20: Reference image for battery safekeeping bag

2.2.8 How to prepare the drone for harsh environments?

There are several ways to prepare the drone for harsh environments, the batteries as mentioned before can simply be interchanged with fully charged close to room temperature batteries, but other components may require more complex solutions. For the most part the components are chosen based on the specification that they can survive in a minimum temperature of -10°C . This is already a good starting point, but other good practises can be to weatherproof the wiring, close unused ports and of course the more risk heavy of the options utilizing chemical coatings to waterproof components. The ESC's and the power distribution board should be fine given that the sides of the drone frame are closed to prevent possible snow from entering the sides of the frame. Given that they produce their own heat and that they are near one another they will be able to keep themselves warm. For the motors the most exposed components it could be good to design a guard that can protect the propellers as well as provide some warmth for the motors that also doubles as motor protection. Enclosing the entire top of the frame can provide more safety and protection from the cold but it may bring up the issue that the condensation generated from the jetson will free and cause more trouble than it is protecting. Finally, there is the component that will come into interaction with human hands meaning they have to both keep the user warm and the transmitter. The solution for this is simply to purchase specialized glove that can fit both hands and a transmitter as in the image below.



Figure 21: Reference image for the transmitter gloves

2.2.9 Obstacle avoidance logic

The obstacle avoidance logic is activated by the pilot through a dedicated switch on the remote controller. This ensures that the system only takes control when the pilot intentionally enables autonomous assistance. Once activated, the Jetson Orin Nano starts processing the video and depth data from the Intel RealSense D415 camera. The depth image is converted into a simplified detection area in front of the drone. This area is divided into multiple smaller zones, such as left, centre, right, upper, and lower sections. By analysing the measured depth values in each zone, the system can determine whether an obstacle is present and in which direction free space is available. During flight, the Jetson continuously checks the distance values in these zones. If the centre zone is clear, the drone continues its current flight path. If an obstacle is detected in front of the drone, the system compares the surrounding zones to determine the safest avoidance direction. For example, if the left side is obstructed but the right side is clear, the Jetson sends a command to the flight controller to move the drone to the right. If the right side is obstructed but the left side is clear, the drone moves to the left. If both sides are blocked but the upper zone is clear, the drone may climb to avoid the obstacle. These movement commands are sent from the Jetson to the Cube Orange+ flight controller through MAVLink, while ArduPilot remains responsible for stabilisation and low-level flight control.

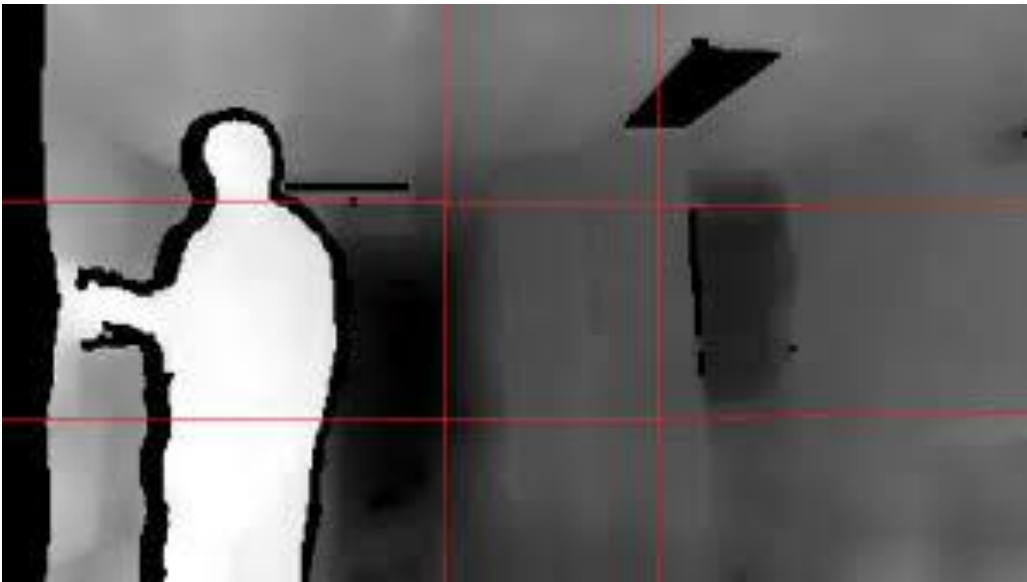


Figure 22: Example of how the grid could look like

If the complete forward detection area is obstructed, the drone first slows down or stops to prevent collision. It can then move backwards until a safe distance is reached. After this, the drone can slowly rotate or scan the environment until a clear path is detected. Once a safe direction is found, the drone continues its flight using the newly selected path. If no safe route can be found after a set number of attempts or within a predefined timeout, the system alerts the pilot that manual intervention is required. In this situation, the pilot can disable the obstacle avoidance switch and resume manual control of the drone. This approach keeps the pilot in control while allowing the companion computer to assist with real-time obstacle detection and avoidance. The logic for the obstacle avoidance would start after the pilot triggers the required switch from then on, it will run the required code and then start avoiding objects as it's flying. As to how this can be approached is by taking the video feed from the camera and combining its depth information into a useful rectangle. This rectangle can then be evenly divided into multiple squares. If all squares on the left withing a certain distance are obstructed the jetson will be able to provide feedback to the drone to make a manoeuvre to the left same goes for the other directions. If the entire drone is obstructed it should move back until safe distance is reached and then attempt a rotation until a clear path is found. If it cannot find it the drone will alert the pilot that manual intervention is required and that the pilot should disable the switch and the pilot will need to resume the rest of the flight manually.

To perform this analysis, the Intel RealSense D415 continuously captures synchronized RGB and depth frames at a resolution of 640×480 pixels. Before obstacle detection is performed, the depth stream is aligned with the RGB stream to ensure that both images represent the same physical locations within the environment. This alignment simplifies future expansion toward object recognition and computer vision applications. The captured depth image is then processed using the Intel RealSense SDK and the Python library `pyrealsense2`. To improve measurement reliability, a spatial filter is applied to reduce sensor noise while preserving object boundaries. Additionally, a hole-filling filter is used to compensate for missing depth values that may occur due to reflective surfaces, poor lighting conditions, or limitations of the depth sensor itself. The result is a cleaner and more consistent depth map that can be used for obstacle detection. Once the depth image has been filtered, it is divided into a grid consisting of multiple equally sized regions.

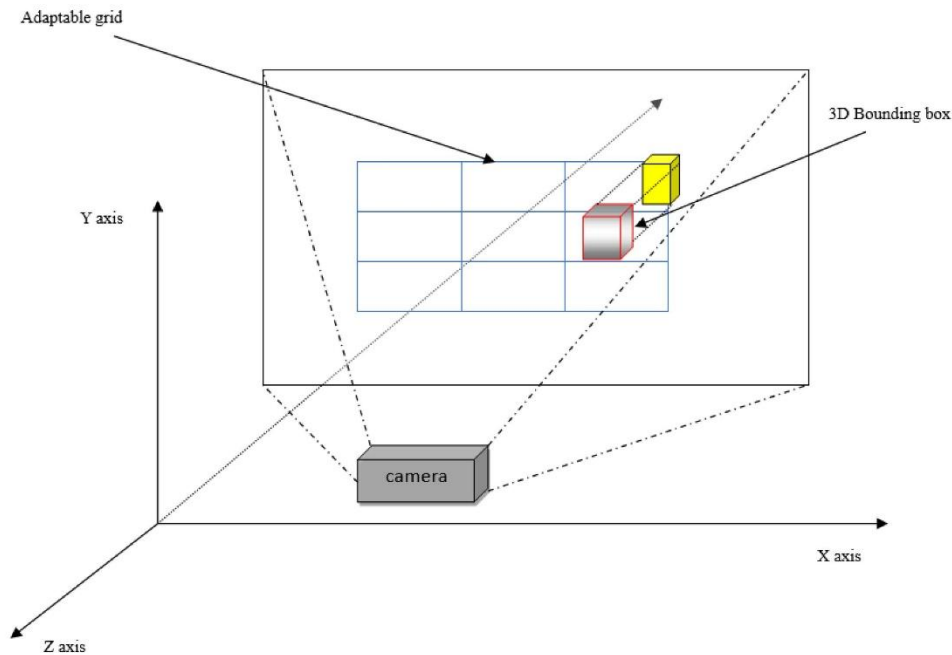


Figure 23: visualisation of the obstacle avoidance concept (Jutai, 2023)

Rather than analysing every individual pixel, the obstacle avoidance algorithm evaluates each region separately. For every region, a representative depth value is calculated using the median of all valid depth measurements within that area. The median value is used instead of the average to reduce the influence of outliers and noisy measurements. The calculated depth values are then compared against a predefined safety threshold. Regions containing objects closer than the threshold distance are marked as occupied, while regions exceeding the threshold distance are considered free space. The occupancy information from all regions is combined to create a simplified representation of the environment surrounding the drone. Based on this occupancy map, the avoidance logic determines the safest available direction. If the left side contains fewer occupied regions than the right side, the system selects a leftward avoidance manoeuvre. Similarly, if the right side provides more free space, a rightward movement is selected. Vertical movement decisions can be generated when obstacles are detected predominantly in the lower regions of the image. If no safe route is identified, the system defaults to a backwards movement and attempts to reassess the environment before requesting pilot intervention. This processing pipeline transforms raw depth measurements into navigation decisions in real time and forms the foundation of the vision-based obstacle avoidance system developed during this project.

2.3 Software setup and hardware connections

Now that all the components have been determined and the frame is fully built it is time to connect them all together and get the drone working. For this process the documentation from the Ardupilot community as well as the Px4 community was used to make sure there were no mistakes in the build of the drone like short-circuits or other possible problems that can break the drone. To have a structured way of working the plan of action was to test and setup each component individually and then moving on to each drone frame to make everything work on itself. Starting with the most important part of the setup the first-time setup.

2.3.1 Flight controller first time setup

To get started with setting up your flight controller you will need to install a GCS (ground control station) program for setup. When starting with this project Mission planner was used but this ended up being interchanged with QGroundControl. Executables for these can be found on their respective sites but setup for the two is identical with the only factor that differs is a slightly different UI. After the installation is complete a reboot is recommended to make sure that all the setup is done correctly. Once rebooted the FC of choice can be plugged into the user's computer. If it is the first time setup of the GCS will prompt the user to install an operating system on the FC, for this project Ardupilot was used since it has better integration for ELRS. So once prompted choose that as a firmware and wait for the installation procedure to be installed. If the beeper is installed (or the GPS with a beeper) and inside the GCS, the user can now reconnect. It is important that if the user wishes to update the firmware of the flight controller that all the previous setups will be removed so it is best to keep a parameter setup to avoid having to redo everything that was done before. After that the top right of the GCS can be setup as in the image below and the user can move on to setting up the sensor calibration. If using QGroundControl the software will automatically attempt a reconnect with the FC. After the auto connect is completed the user will be required to input a bit of information regarding the drone specifications.

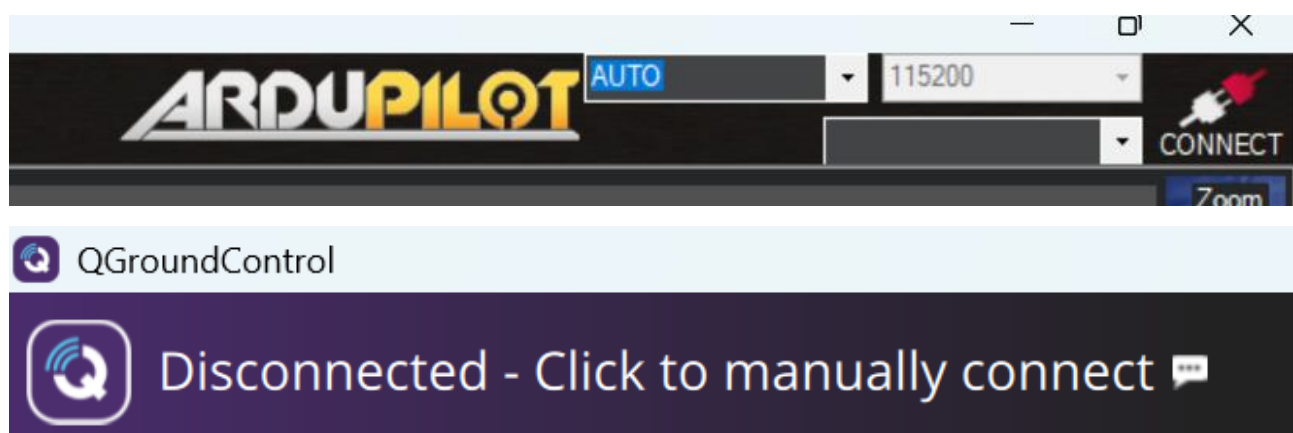


Figure 24: Ardupilot and QGroundControl connection demonstration

The first thing asked of the user after connecting will be to select the drone frame that will be used. For this project the simple quadcopter frame will suffice (image below for reference). This is best reference for the utilized frame after this step it is time to setup the sensors.



Figure 25: Reference for the frameoptions

2.3.2 Sensor calibration

Sensor calibration is a step with incredible importance since this will determine how well the drone will fly and how stable as well as setting up the GPS. To get this started the user needs to head over to the Accel calibration. Then the image below will be visible. It is recommended to press each button and then follow the required steps as requested by mission planner making sure to keep the arrow on the flight controller into the requested direction. For the second and third button the drone simply needs to be level and then this part of the sensor calibration is completed, and the GPS setup can be started.

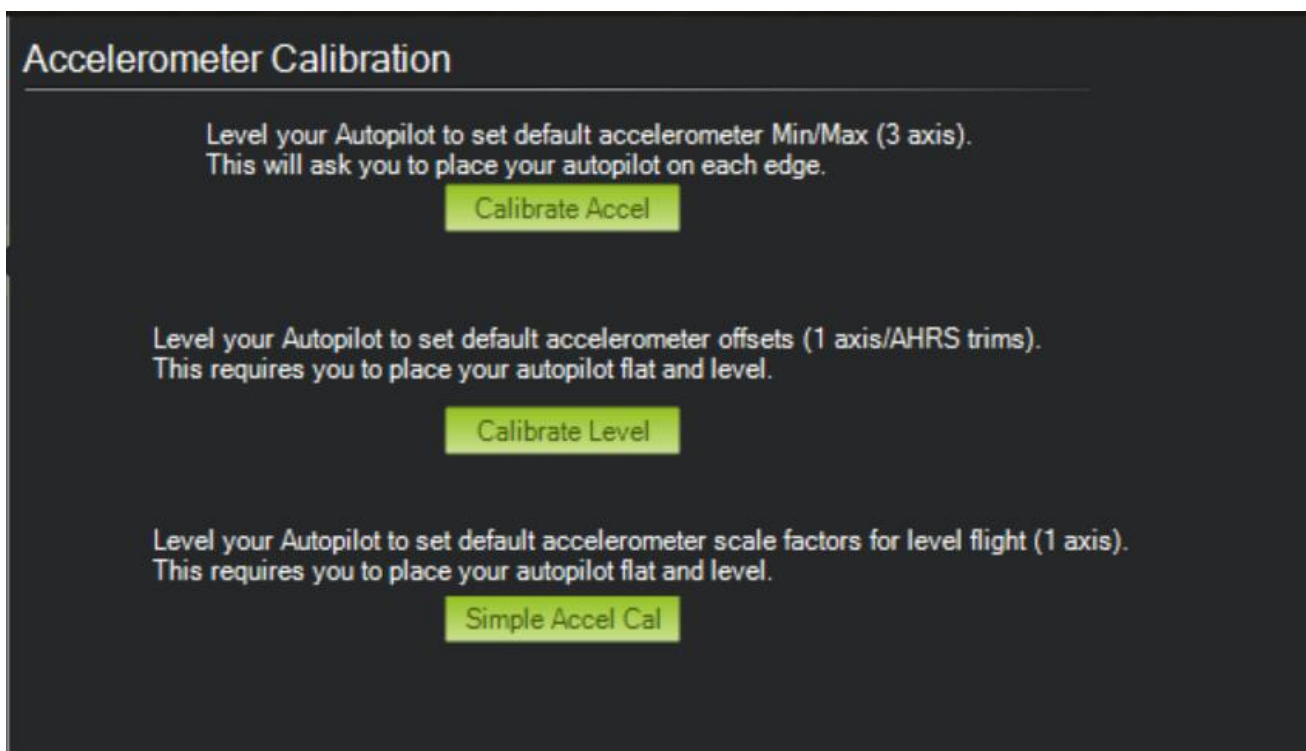


Figure 26: Sensorcalibration menu

Before the GPS setup can be started the GPS module will need to be plugged in for both flight controllers it is possible to use the GPS 1 port since this is often the better supported of the two. After this the Compass tab can be opened and the GPS setup and calibration can be started. Firstly, it is important to make sure that the external compass has top priority since it offers better accuracy since it has less component interference. Clicking the up and down arrows can switch out the priority. Then it is important to note that there should only be 2 GPS's visible if there is only one check the connections or press the reboot button. If there is a third check the "Use compass 3". After this the start calibration step can be done, and it is important to align the arrows of the external GPS with the arrow on top of the flight controller. Then move and tilt the flight controller until both the MAG 1 and MAG 2 bar are filled all the way. The flight controller will indicate the step is completed by giving a beep. After this is is important to reboot and go to the DATA page to check if the FC is still levelled out and the GPS location is accurate.

Compass Priority

Set the Compass Priority by reordering the compasses in the table below (Highest at the top)

Priority	DevID	Bus Type	Bus	Address	Dev Type	Missing	External	Orientation	Up	Down
1	658953	I2C	1	14	IST8310	☐	☒	Yaw270	⬆	⬇
2	1313809	I2C	2	12	AK09918	☐	☐	None	⬆	⬇

Do you want to disable any of the first 3 compasses?

☒ Use Compass 1
 ☐ Use Compass 2
 ☒ Use Compass 3
 Remove Missing
☐ Automatically learn offsets

A reboot is required to adjust the ordering.

Reboot

A mag calibration is required to remap the above changes.

Onboard Mag Calibration

Start
Accept
Cancel

Mag 1

Mag 2

Mag 3

Fitness Default Relax fitness if calibration fails

Large Vehicle MagCal

Figure 27: GPS order reference

2.3.3 Transmitter and receiver setup

2.3.3.1 Transmitter

This step will require setup on the transmitter, the receiver and the flight controller. Starting with the transmitter if it is not new or the settings have not been altered this step can be ignored. (Please note that in alternate remote software options or future iterations this setup may vary slightly) Powering it up for the first time not much will be able visible on the starting screen with fresh settings. Pressing the roller button will provide a few options to get started press the manage model's option. Then the option NEW will be visible in the top, you will then get to choose to create a new model. This can either be done from an existing one or a blank one. To keep setup easy the best option is to go blank since this allows more flexibility. Having done this the start screen will be visible again and pressing the MDL button will bring us to the required settings. Here the name can be changed to easier remember the newly created custom profile. Now press the next page button several times until the mixes page can be found. On here channel 5 to 7 will be added for use with the drone. Channel 5 will be used for arming and disarming, channel 6 will be used for switching between flight modes channel 7 is a secondary arming switch or emergency motor stop. Setup for all of them is the same by simply providing a name to easier remember the setup. The user can now press or navigate to the source button to bind a switch from the controller to this. This can be any switch or button but for ease of use and what is typically used in the industry is the top right for arming and top left for switching modes. By toggling the used switch, the software will immediately move to the requested switch, and the switch can now be selected. Be sure to keep the arrow position in mind since this will determine what position the triggering will occur. Repeat this step for the other switches as well, once that is done click on each channel and insert after doing the same as before but now with the position you would like to label as off. Finally, you can add the widget settings by clicking on the screen when it's empty and pressing add widget later selecting outputs this will show what position the switches are set on. Before turning off the transmitter, it is always best to check that the backpack is enabled in the flight controller by going into settings and moving all the way to hardware and setting External RF to normal. After this everything that needs to be done on the remote is complete!



Figure 28: Transmitter final screen

2.3.3.2 Receiver

To get started a cable from the Pixhawk/orange cube + cable set will need to be split in two and stripped be sure before doing this that the cable fits in the telemetry ports! After this is confirmed, the user can solder the cable as shown in the diagram below.

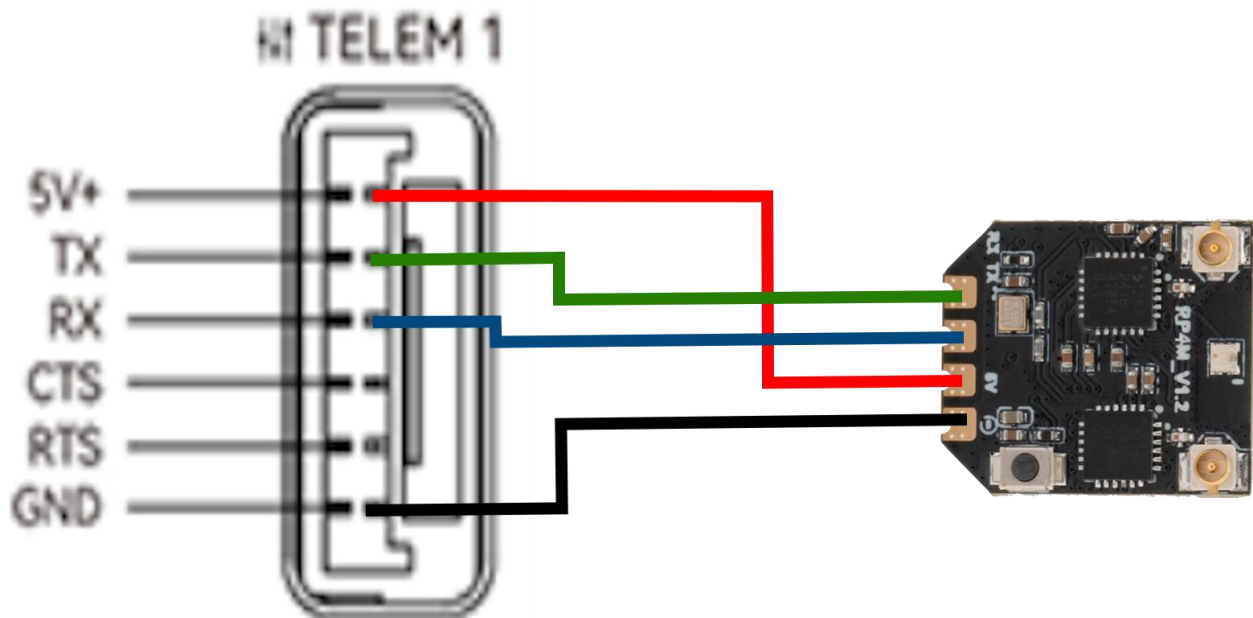


Figure 29: Receiver connection schematic

After this is done the telemetry module can be plugged into the flight controller so it can be started. After it's plugged in for the first time the led will be blinking yellow intermittently this means that it is looking for a transmitter. Going into the transmitter settings into model and then by navigating to External RF and making sure that the mode is set to CRSF (crossfire) the binding process can be started. By going to the binding button, the process will be started. If this is successful, the led will change from a solid blinking yellow LED to a solid blue LED to indicate a solid connection. Should it be that it took to long to setup and the receiver is now switching colours it means that Wi-Fi mode is enabled and simply replugging the receiver or unplugging and replugging the battery will restart it back into binding mode. Alternatively, holding the button on the receiver will also work. With this the receiver setup is complete, and the flight controller setup can start.

2.3.3.3 Flight controller

For this step it is important to note in what port the receiver is plugged into, this is important since most of the flight controller settings are bound like so serialX with X standing for the number of the serial port. Keep in mind that this value is also different between the Pixhawk C6 and the Orange cube please be sure to check the documentation from the respective sites as to what this should be set as. The port most suitable for the RC communication is the telemetry two port as of writing this is digitally labelled as serial 2 for both. After this is complete the radio should now be visible and movements from the remote should be reflected inside mission planner. If this is not the case check the wiring from the last step as well as the setup again to make sure that there is nothing wrong with those. If this does work the calibration can be ran following the steps as that the program requires. When this is complete the previously setup channels can be enabled in the full parameter list starting with channel five. By looking up "ch5" in the full parameter list and then adjusting its value to arming and disarming the arming can now be done for the drone using the remotes channel 5. Repeat this step for the other channels to set up those as well to work accordingly, making sure to use channel 6 for FTL mode switching. After that a reboot can be good to save and write all parameters. Once back head to the FTL mode page and here the choice can be made for which flight modes that should be desired. The first one is best to leave at stabilize since this is good for stable flight. Loitering can be good for needing to prepare for the obstacle avoidance and acro is a good one for testing how stable the drone flies without gyro support. With this the remote is fully setup and ready for flying.

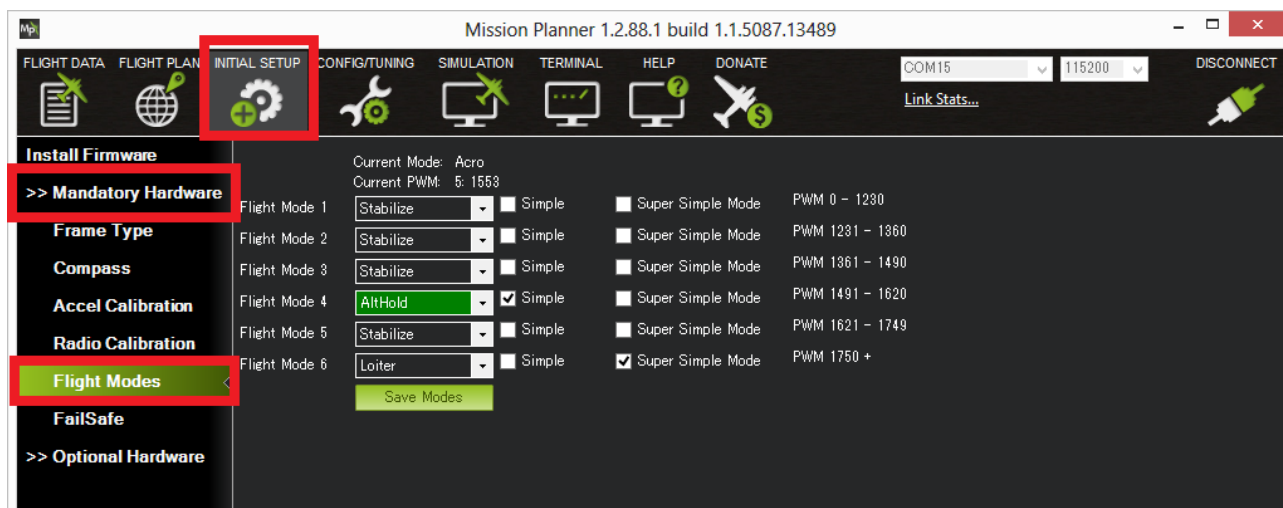


Figure 30: Flightmode settings

2.3.4 Motor and ESC setup and calibration

Connecting the motors and ESC's is straightforward. To correctly do this the manufacturers documentation can be consulted for correct utilization and connection of the ESC's and the motors. This can be as seen in the image below. The image shows how it can be done for one motor and ESC set, replacing the battery with the power distribution board for the polar connections and the receiver with the arduino for testing or eventually the flight controller for final setup. Before heading to the flight controller, it is important to make sure that the motor direction is correct now that it is still able to be easily adjusted. To do this the Arduino can be installed with the testing code and making sure not to have the propellers installed they can be turned on one by one to match the motor directions with the schematic below. With this complete the hardware steps are done, and the calibration can be done in the flight controller.

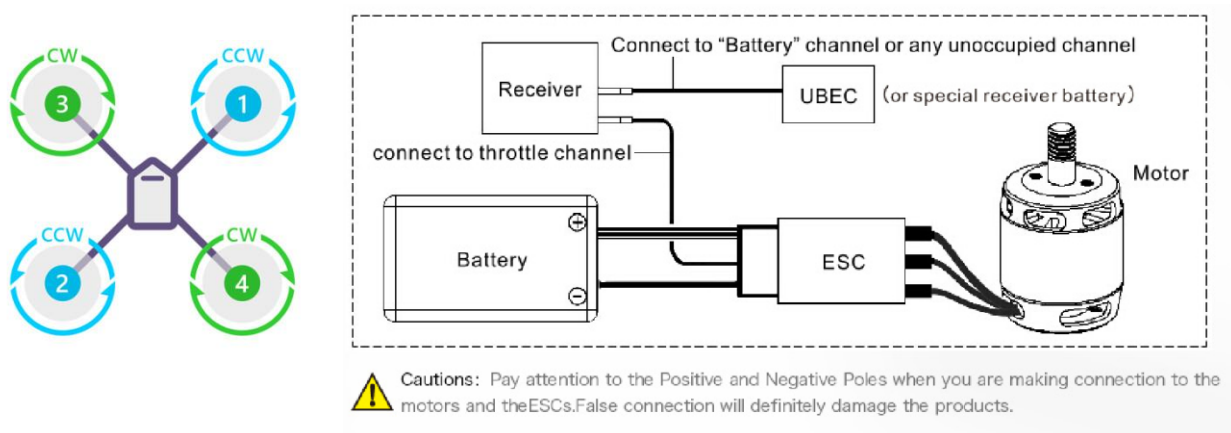


Figure 31a: Motor direction guide/ Figure 31b: Esc first time setup

Opening mission planner there will be a few settings that will require changing most importantly being the ESC auto calibration making sure that this is set to the value "0" in other words at normal startup. Then it is important to test and find the correct ESC type. In the preformed research the best matching ones for the test frame was one shot125 as for the new frame it is DShot600. With this done the calibrate ESC button can be pressed to commence calibration as described inside of mission planner. If all steps go well the ESCs should perform a small musical tune to indicate a correct setup this is also true for the Arduino's. For the new frame it is also important to note that once powered it will provide a beep with a 1 second interval this is normal and simply indicates that the ESC cannot start the motors which is logical since the flight controller does not allow it without the safety switch enabled. If everything is setup correctly the drone should be able to have the motors spin after disabling the safety switch and arming the remote once more making sure to enable the arming switch on the transmitter as well with this completing the ESC and motor calibration and hardware connection.

2.3.5 Jetson setup and flight controller communication

2.3.5.1 Jetson and depth camera setup

The Intel RealSense D415 depth camera is connected to the NVIDIA Jetson Orin Nano through a USB 3.0 interface. To enable communication between the Jetson and the camera, the Intel RealSense SDK (librealsense) must be installed. This software package provides the required drivers, utilities, and application programming interfaces (APIs) for accessing both RGB and depth image streams. Installation is performed directly on the Jetson after the operating system has been configured. Once the SDK has been installed, the camera can be verified using the RealSense Viewer application. This utility provides a real-time visualization of the RGB and depth streams and confirms that the camera is operating correctly. Successful detection of the camera and the display of valid depth measurements indicate that the hardware and drivers have been configured properly. To support software development in Python, the pyrealsense2 package is installed. This library provides access to the RealSense API and allows Python applications to communicate directly with the camera. The developed software initializes a RealSense pipeline and configures simultaneous depth and colour streams at a resolution of 640×480 pixels and a frame rate of 30 frames per second. Before processing, the depth image is aligned with the colour image to ensure that both data streams correspond to the same physical locations in the environment.

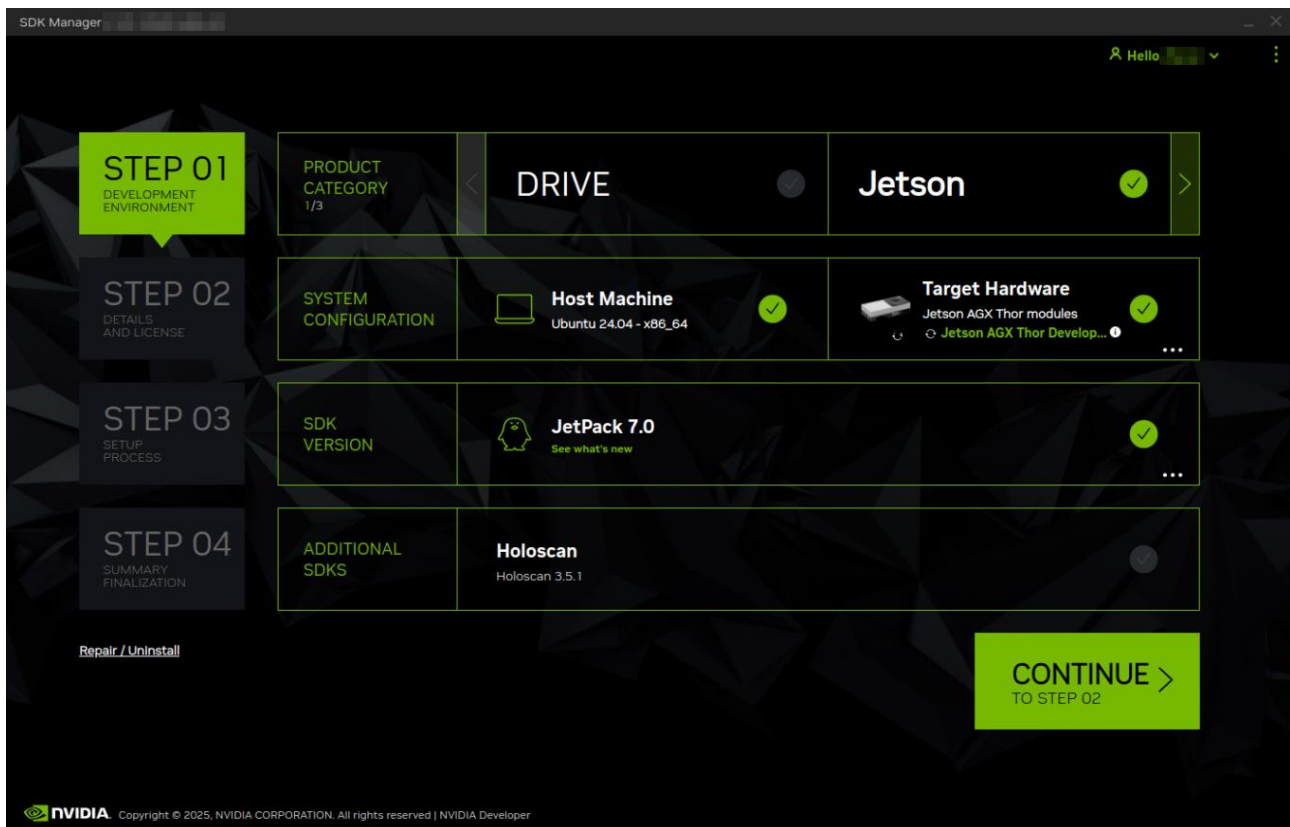


Figure 32: Nvidia jetpack SDK

To improve the reliability of depth measurements, spatial filtering and hole-filling filters are applied to the captured depth data. These filtering techniques reduce sensor noise and compensate for missing depth values, resulting in a cleaner and more accurate depth map. The processed depth image is then converted into a numerical format that can be analysed by the obstacle detection algorithm. By providing accurate distance measurements in real time, the Intel RealSense D415 forms the primary sensing component of the obstacle avoidance system and enables the Jetson Orin Nano to identify obstacles and determine safe navigation paths. To get started with setting this up the Jetson will require a software image. The Jetson Orin Nano uses NVIDIA JetPack, which is based on Ubuntu Linux and includes all required drivers, CUDA libraries, and hardware acceleration tools. This can be done by following the official guide from Nvidia utilizing the Nvidia SDK manager on PC. (Version utilized for this project was 6.2 be carefull with SSD size since this may result in no boot). After installation, the system is updated using the Linux package manager to ensure that all software packages and security updates are installed. JetPack also provides access to CUDA, cuDNN, TensorRT, and other hardware-accelerated libraries that can be used for future artificial intelligence applications.

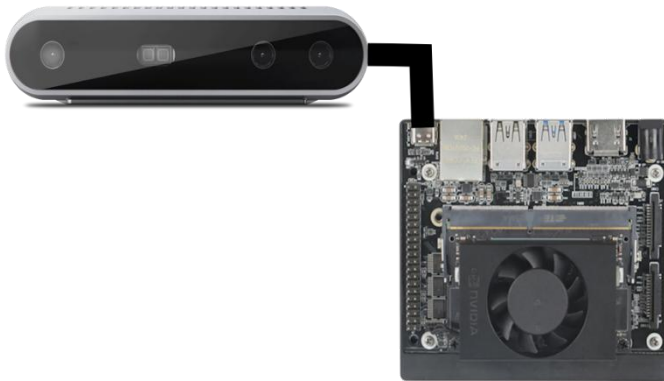


Figure 33: How to connect the jetson with the Intel D415

The Intel RealSense D415 depth camera is connected to the Jetson through a USB 3.0 interface. To enable communication with the camera, the Intel RealSense SDK (librealsense) must be installed. This SDK provides drivers, camera utilities, and application programming interfaces (APIs) for accessing depth and colour image streams. Once the SDK is installed, the camera can be tested using the RealSense Viewer application, which allows live visualization of depth and RGB data. Successful operation confirms that the camera is detected correctly and that depth measurements are available. Python support is enabled by installing the pyrealsense2 package. This package allows software running on the Jetson to access depth frames directly from Python. A typical application initializes the camera pipeline, enables depth streaming, and continuously retrieves frames. The depth image can then be processed to determine obstacle distances and free navigation paths.

2.3.5.2 GPIO Configuration and flight controller setup

The Jetson Orin Nano provides General Purpose Input/Output (GPIO) pins through its 40-pin expansion header. These pins can be used to communicate with external hardware such as LEDs, switches, sensors, or status indicators. GPIO functionality is enabled through the Jetson GPIO Python library. This library offers an interface like the Raspberry Pi GPIO framework, simplifying software development. After installation, GPIO pins can be configured as inputs or outputs directly from Python. Input pins may be used to monitor switches or external signals, while output pins can control LEDs or other peripheral devices. In the context of this project, GPIO functionality may be used for debugging, status indication, or interaction with additional sensors. Combining GPIO control with RealSense depth processing allows the Jetson to interface directly with both sensing hardware and the flight control system while maintaining real-time obstacle avoidance functionality. Firstly, a cable should be created as shown in the image below. There should already be one available but keep in mind not to connect the 5v when the orange cube is powered on to avoid unnecessary problems but for testing communications it is useful to have a power source for it. Another way to setup communication between the two is by simply utilizing a usb connection between the orange cube and the jetson. The image shows telemetry one but Telemetry 2 or GPS2 will work as well. To setup the communication the same steps can be utilized for setting up the receiver for setting up the communication between the Jetson and the flight controller.

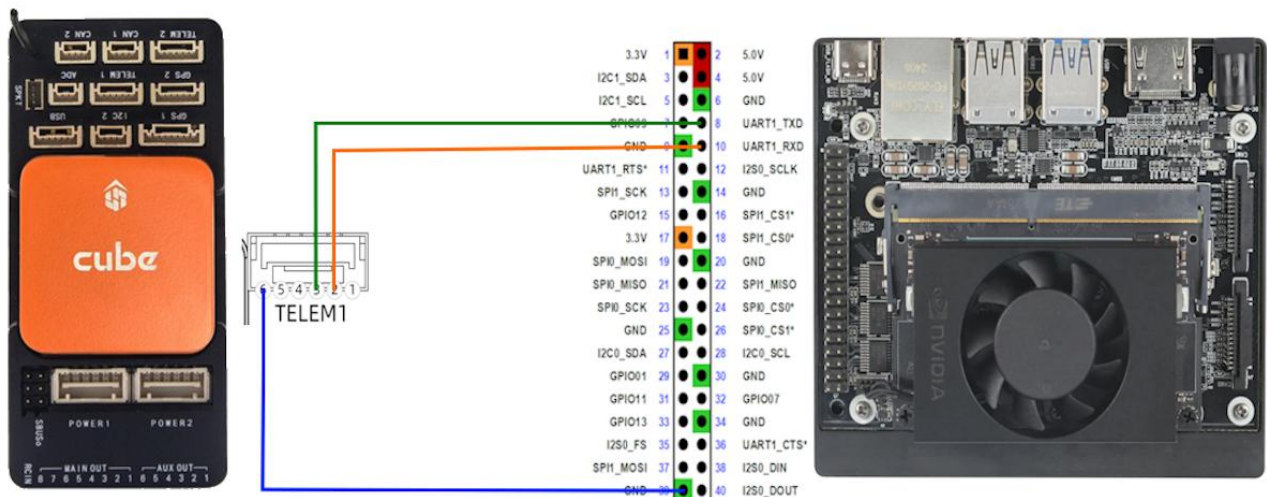


Figure 34: GPIO connection with the Jetson Orin nano

2.3.6 Final schematics

With everything setup and connected it is always good to verify that everything is connected as it should be it is time to verify as well as add some components that will be required for full operation of the drone. The centre of this project is the Orange cube+ to this everything is connected. Firstly, and easiest to verify is the GPS this should be plugged into the GPS one port. Another easy one is the receiver this should be plugged into either one of the telemetry ports or the GPS2 port making sure to do the necessary alterations to make it all work. With that out of the way it is important to go over all motor connections making sure they are still correct since last testing them and if nothing got loose, same goes for the ESC's and their connection to the PDB (power distribution board). Also on the power distribution board is the power for the Jetson Orin nano which needs its Voltage Step Down converter in between to make sure that it's not trying to run on the 22 volts provided by the battery. From this it is also good to check the camera USB cable to the Jetson making sure it is plugged into the Jetson. Optionally there is the Telemetry modules they follow the same logic as the receiver and can be plugged into the remaining open ports. On the following pages are useful schematics to check for confirming the drone connections.

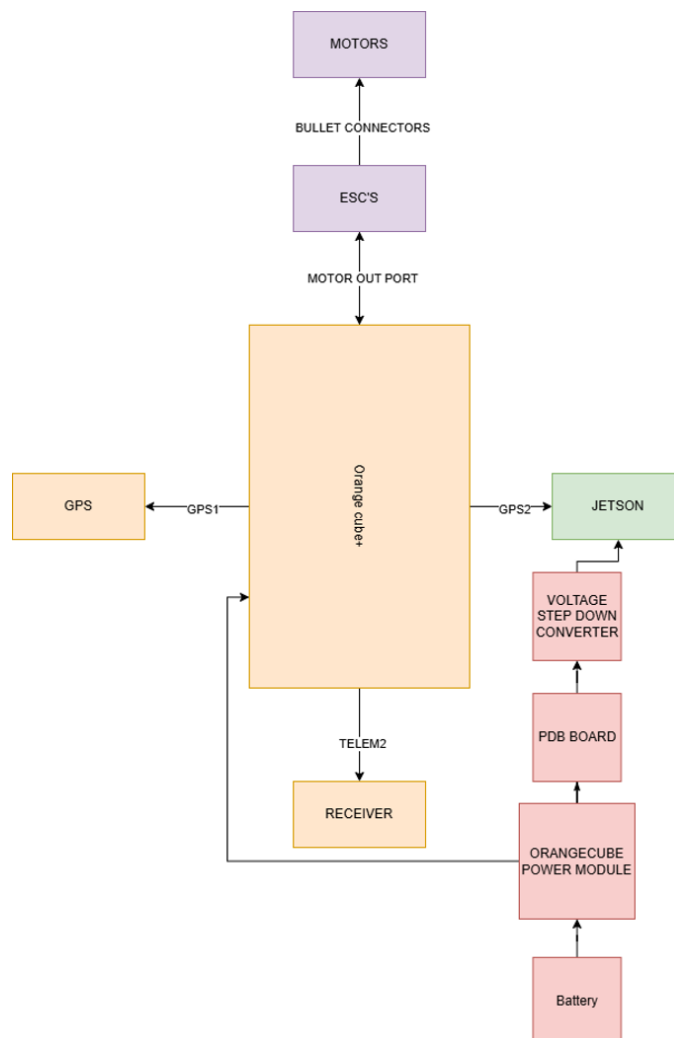


Figure 35: Full schematic flowchart

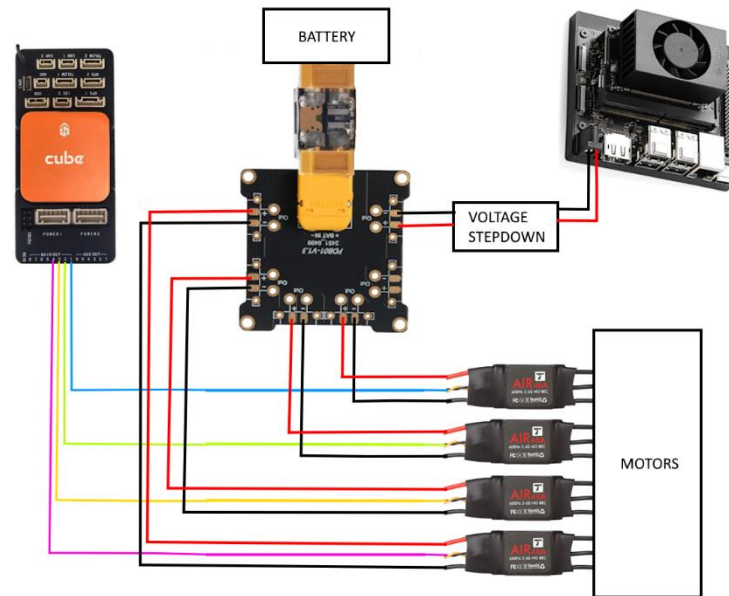


Figure 37: Power Management schematic

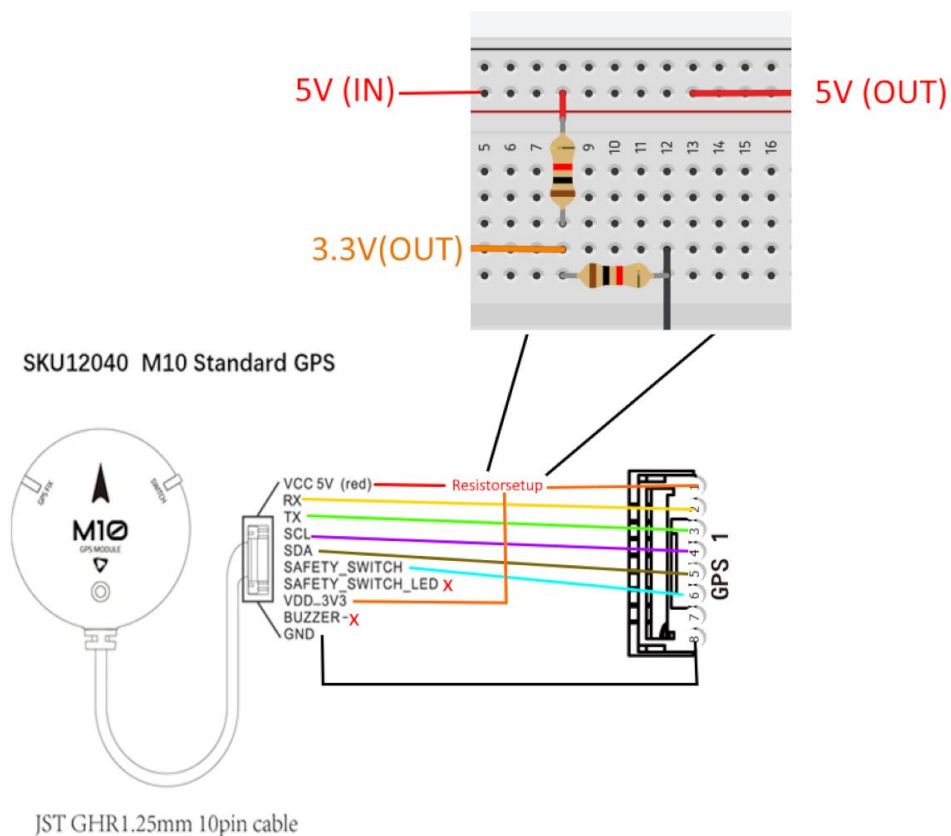


Figure 36: GPS wiring diagram

2.4 Testplan and flighttesting

2.4.1 Phase 1 full system test

For this stage of the test both the test frame and the new frame were simply place upon the test and inspected for possible issues with wiring or voltages. After making sure nothing is wrong and the ground control program on the laptop is not providing errors the user can now disable the safety system (making sure the propellers are removed) and test the system functionality. This can be done by simply moving the throttle up slowly and seeing how the drone reacts. The different modes can also be tested by placing the switch in several different positions to confirm that the modes change based upon the switch position.



Figure 38: Photo taken during motortesting on desk

2.4.2 Phase 2 simple outdoor propeller test

Now that it is confirmed that the drone setup is proper and fully functional as expected it is time to install the propellers according to the image below and to take the drone outside to start the propeller test. For the propeller installation it is important to make sure that it is correct and to doublecheck the motor direction as well to avoid the drone being unable to fly or worse crashing. If this is done correctly and doublechecked the drone can be turned on with the safety switch and it is time to raise the throttle. It is important to not raise it too high because the goal of phase 2 is to determine if the drone has any kind of tilt and to triple check that all motors are spinning in the correct orientation (should it have been missed before). If all goes as expected turn the throttle down all the way and study the frame.

2.4.3 Phase 3 obstacle avoidance test

After the previous testing steps have been completed the jetson can be connected to the drone to start the obstacle avoidance test. For this the drone will be flown manually to with the goal to avoid a premade obstacle path. The obstacle path will have a set start which will also function as the finish line. This test will require two users referred to as the pilot for the person that is in control of flight and the navigator for managing QGroundControl. The pilot will manoeuvre the drone into the correct position then place it into loiter mode so that they can signal to the navigator to start drawing the path. This can then be marked as the starting point in QGroundControl, from this a basic path can be set for the drone to follow making sure that the starting point is connected to the endpoint. Completing this the circle and making sure the path is marked correctly the test can be started by starting autonomous flight the pilot has to make sure to be ready at all times to take over, the navigator has to do the same making sure that the drone does not steer off the preset path and that the drone does not send it's emergency message to alert the pilot that manual flight has to be resumed and monitor the camera feed to again make sure there are no troubles in flight. If all goes well and this test is running several times ran it can be determined that the drone is functional as required.

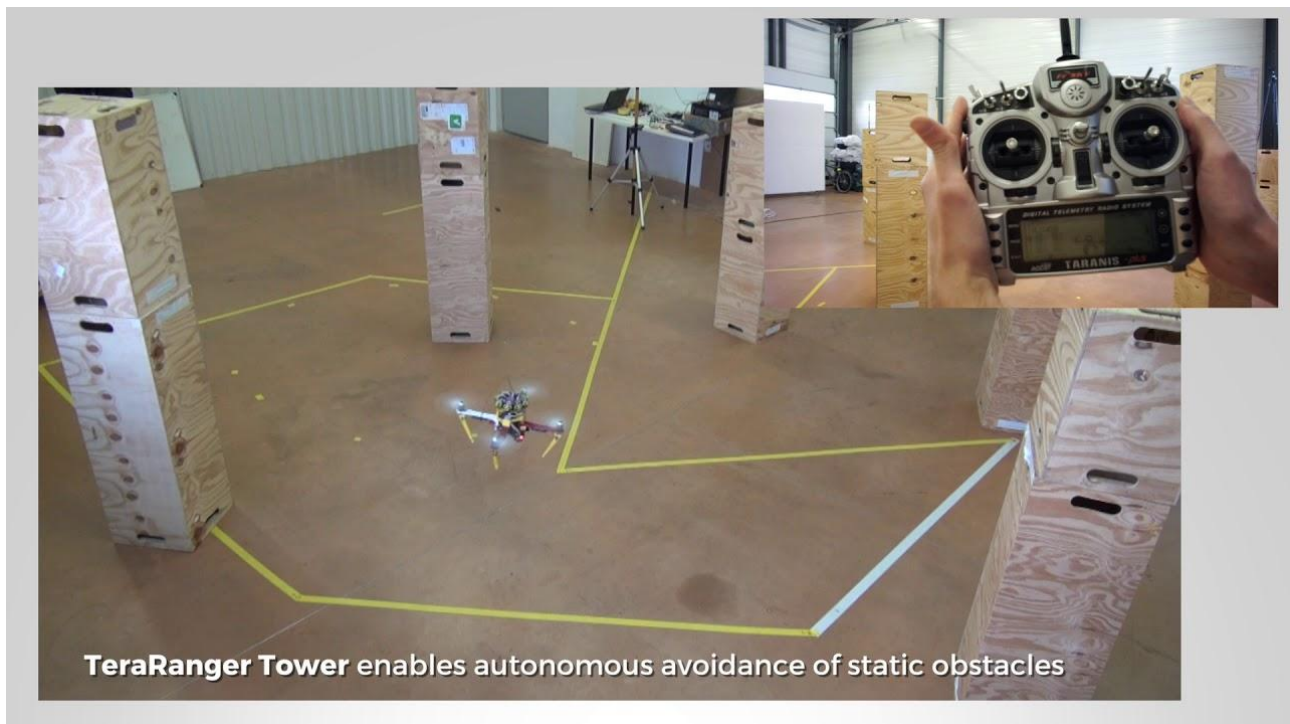


Figure 39: Example of a possible obstacle path (Terabee, sd)

3 Fase 3: Results

3.1 Drone frame and setup

The developed UAV platform achieves a Technology Readiness Level (TRL) of approximately 2–3, while still allowing room for further development and optimization. From a hardware perspective, the drone platform is fully assembled but not fully build as promised. The system responds to user input, confirming near correct configuration of communication protocols and flight control parameters. Manual flight operation is therefore supported, including safe arming and disarming procedures, although full autonomous flight remains to be completed. The image below demonstrates the final state the drone was left in after completion of the project. All the required hardware is there the esc's, and the motors are wired together and then connected to the orange cube and calibrated to do manual flight as intended. The battery also has its holder for making sure it does not fall out during flight but so it also remains easy to replace. The same can be said for the jetson, the depth camera and the flight controller. All are mounted inside the frame so they can easily be taken out and replaced for emergency situations. The flight modes themselves still need further calibration as Loitering had some troubles remaining stable in the air but otherwise everything is there to continue working on the project and elaborate on the software prototypes.

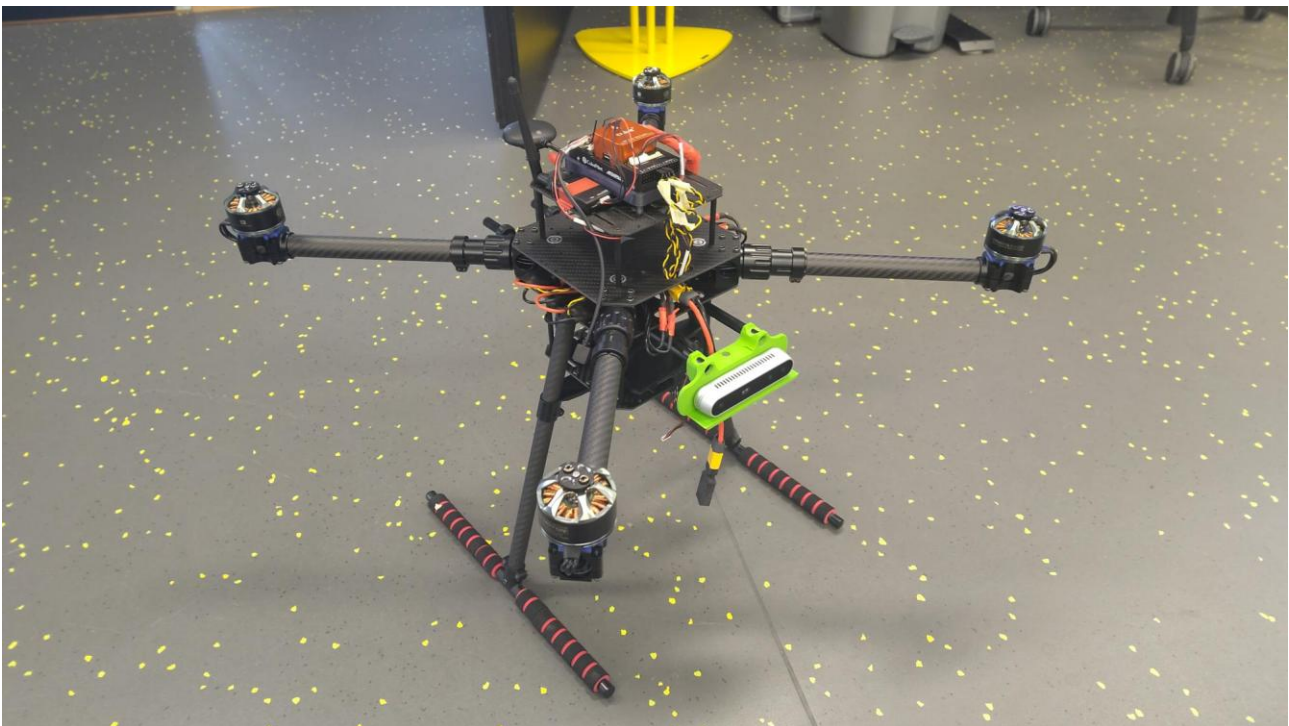


Figure 40: Drone 008 almost fully assembled

3.2 Autonomous flight code

In parallel with the UAV platform, a companion computing system based on the NVIDIA Jetson Orin Nano was required to be created. This system was meant to cooperate with the flight controller and can execute computer vision algorithms for object detection and avoidance. The Jetson environment supports real-time image processing and can perform basic obstacle avoidance logic using the intel D415 as camera input. The implemented algorithm is a simple prototype that can detect basic obstacles and determining directional avoidance actions, demonstrating that the perception pipeline from data acquisition to decision-making has a prototype. The prototype of this code and a basic prototype is installed on top of the jetson Orin nano and can be ran and tested by hooking up the jetson to a monitor and keyboard and opening the testing environment. This is one of the two prototypes, both are provided in the end of this document should there be interest into using it in the future.



Figure 41: Demonstration of the eventual obstacle avoidance code

The second prototype is a communication test with the provided hardware, such as the Pixhawk C6, confirm that telemetry data exchange between the Jetson and a flight controller is feasible. The Jetson can retrieve flight-related data, including orientation and flight mode, validating that the groundwork is there for communication. Despite these successful subsystem validations, full system integration has not yet been achieved. Communication between the Jetson Orin Nano and the Cube Orange+ remains incomplete. While the Jetson can read the Orange Cube data it cannot write actions, so the software interface required for reliable command transmission is not yet operational. As a result, the AI-based obstacle avoidance system cannot currently influence the UAV during flight. Nevertheless, the project establishes a modular and framework in which both the UAV platform and the AI subsystem are individually validated. The combination of a functional drone system, a working perception pipeline, and partially verified communication pathways demonstrates that the overall concept is technically feasible. The remaining challenge lies in achieving robust integration between these subsystems to enable fully autonomous operation.

4 Discussion

4.1 Technical reflection on the results

During the development of the UAV platform, several technical challenges were encountered that significantly influenced the progress of the project. While several issues were successfully resolved, others required modifications to the original approach. The most significant challenge was the integration and validation of the complete hardware platform within the available project timeframe. A meaningful situation arose during the hardware integration phase when the assembly and testing of the drone platform took considerably longer than anticipated. During the research phase of the project, it was expected that all required components could be acquired and integrated within a relatively short period. However, a significant amount of time was lost due to sourcing and shipping delays. Although all components were eventually obtained through European suppliers, the delay reduced the available development and testing time. To mitigate this issue, an existing test frame was utilized to begin software development and subsystem testing before the final platform was fully assembled. Reflecting on this situation revealed that hardware availability can have a substantial impact on project planning and development. While the use of the test frame allowed progress to continue, additional issues were discovered during testing that required further investigation. By requesting assistance from supervisors and peers, the root cause analysis of the test frame could be diminished whilst working on the new test frame as well. This experience highlighted the importance of risk management and parallel development strategies during complex engineering projects. Another challenge occurred during the software configuration of the NVIDIA Jetson Orin Nano. During the implementation phase, several days were spent diagnosing persistent boot issues after installing JetPack. Initial investigation focused on software configuration and installation procedures, as no error messages were generated during the flashing process. After extensive troubleshooting, it was determined that the installed SSD did not meet the practical storage requirements of the selected JetPack version. Although the official documentation warns against using storage devices smaller than 256 GB, the selected 256 GB SSD still proved insufficient in practice. This resulted in repeated installation attempts before the actual cause was identified.



Figure 42: Assembly x650 frame

The most important conclusion from this experience is that hardware and software requirements should be validated beyond the minimum specifications stated in documentation. Future projects could reduce troubleshooting time by selecting storage devices with additional capacity margins and performing preliminary compatibility checks before software installation. Applying this alternative approach resulted in a successful JetPack installation and stable operation of the Jetson platform. A further challenge was encountered during the integration of the Holybro M10 GPS module with the Cube Orange+ flight controller. During the design phase, it was assumed that the available ports on the flight controller shared a similar pin configuration. Upon delivery and inspection of the hardware, it became apparent that the GPS port configuration differed from expectations due to the integration of an external buzzer and safety switch system. Consequently, the GPS module required modification and rewiring. Additionally, a voltage divider circuit had to be implemented to provide the 3.3 V signal required by the safety switch. This experience demonstrated the importance of thoroughly verifying hardware interfaces before component selection and procurement. Future implementations would benefit from a complete review of connector pinouts and electrical specifications during the design phase. Despite the additional hardware modifications required, the GPS and safety switch systems were successfully integrated and operated correctly on the final platform. The final major challenge involved achieving stable flight performance. During testing on the initial frame, the drone repeatedly performed an uncontrolled backflip immediately after take-off. Investigation suggested that the root cause was related to ESC calibration parameters not being correctly stored within Ardupilot. As a result, motor outputs were not behaving as expected during flight. To address this issue, an alternative validation approach was implemented. The propulsion system was first configured and tested using PX4 firmware on a Pixhawk 6C platform. Once stable flight behaviour was achieved, the resulting motor and calibration parameters were extracted and transferred to the Cube Orange+ running Ardupilot. This approach successfully resulted in stable manual flight on the final airframe. The experience demonstrated the importance of systematic subsystem validation and highlighted that alternative testing platforms can be valuable tools for diagnosing integration issues that are difficult to isolate directly on the target system. Overall, the project demonstrated that hardware integration, software configuration, and system validation present significant challenges in UAV development. However, through iterative problem solving, alternative testing approaches, and continuous refinement of the methodology, a functional drone platform was successfully assembled and validated. These experiences provided valuable insights into the practical challenges associated with developing autonomous UAV systems and contributed directly to the final project outcome. And these are only a few of the problems that were run into during the project time.

4.2 SDG-reflection

The project primarily contributes to SDG 9: Industry, Innovation and Infrastructure. (Vlaanderen, sd) The objective of the project was to investigate the feasibility of a modular UAV platform equipped with a companion computer and depth sensing capabilities for future autonomous obstacle avoidance applications. During the project, a drone platform was assembled, configured, and tested using a Cube Orange+ flight controller, NVIDIA Jetson Orin Nano, and Intel RealSense D415 depth camera. The development and documentation of this platform contribute to technological innovation by creating a foundation that future students and researchers can further expand upon. Using Korthagen's reflection model, a meaningful situation was experienced during the integration of the drone platform and the vision system. The achieved result was a functioning hardware platform combined with a depth-camera-based obstacle detection algorithm running on the Jetson Orin Nano. Although the complete integration of autonomous obstacle avoidance into the flight system was not finalized within the available timeframe, the project successfully demonstrated the feasibility of combining depth sensing, onboard processing, and flight control technologies within a single UAV platform.



Figure 43: SDG list

Reflecting on this experience revealed that developing autonomous UAV systems requires significantly more effort in system integration than initially anticipated. While the individual components functioned correctly, combining hardware, software, communication interfaces, and flight testing proved to be a complex engineering challenge. This highlights an important aspect of SDG 9: innovation is not only driven by new technologies but also by the successful integration and validation of existing technologies into practical systems. The most important conclusion is that the project established a technical foundation for future UAV research and development. The resulting platform provides future students with a documented starting point for further work on obstacle avoidance, autonomous navigation, and advanced sensing technologies. Considering alternative developments, the platform could potentially contribute to additional Sustainable Development Goals. For example, if autonomous obstacle avoidance is fully integrated and validated in future work, the platform could support SDG 11: Sustainable Cities and Communities through infrastructure inspection, environmental monitoring, and support for emergency services. Furthermore, future integration of environmental monitoring sensors could contribute to SDG 13: Climate Action by enabling data collection in remote or difficult-to-access locations. At its current stage of development, however, the primary contribution remains SDG 9 through the creation, validation, and documentation of an innovative UAV research platform. The project therefore contributes to technological knowledge and provides a foundation for future developments that may support a broader range of societal and environmental objectives.

4.3 Value for the workplace

The drone project will always be useful in the work environment because it is something that is versatile easy to adapt to different use cases. And since the Frostbit team is planning to start courses on drones the coming year the frame can be a good starting point to learn about wiring, setup and much more on drones. If it isn't for the drone course, there is of course still the Frostbit research team that can benefit from the drone frame. For example, future interns can continue with this project and bring it to completion. Alternatively future interns can adjust the project goal to for example program pilot tracking based on facial recognition. Or simply make it a drone that can deliver payloads or host a mesh network for drones as previously mentioned in chapter one. The options are truly infinite, though for the far future it may be useful to replace the Jetson Orin nano with something more powerful to keep up to date with modern specifications and possibly better power efficiency. The project as is will be a good starting point for every possible scenario that can be thought of. And the new frame is a much better improvement over the old one.



Figure 44: Two coworkers working on a drone

4.4 Reflection for the X-factor

Several personal learning insights were gained all linked to the PXL-X-factor, highlighting both strengths and pitfalls. Making sure to recognize that the x-factor pillars were demonstrated by being strong, passionate through my motivation to complete the project, enthusiastic to always work on the project, and willingness to help others, while also collaborating effectively in an international environment by exchanging knowledge with colleagues and fellow students. As well as asking for the required help when needed near the end. Additionally, my technical background was applied in a multidisciplinary way by contributing to projects beyond my own. At the same time, identify important areas for improvement within my entrepreneurial and innovative behaviour, as communicating problems was something that needed to become more proactive, structure my planning more effectively and react faster with alternative solutions when challenges arise. These insights show that while I already bring value to the team through collaboration and technical contribution, I still need to further develop my proactive mindset and organizational skills to grow into a complete and more effective professional.

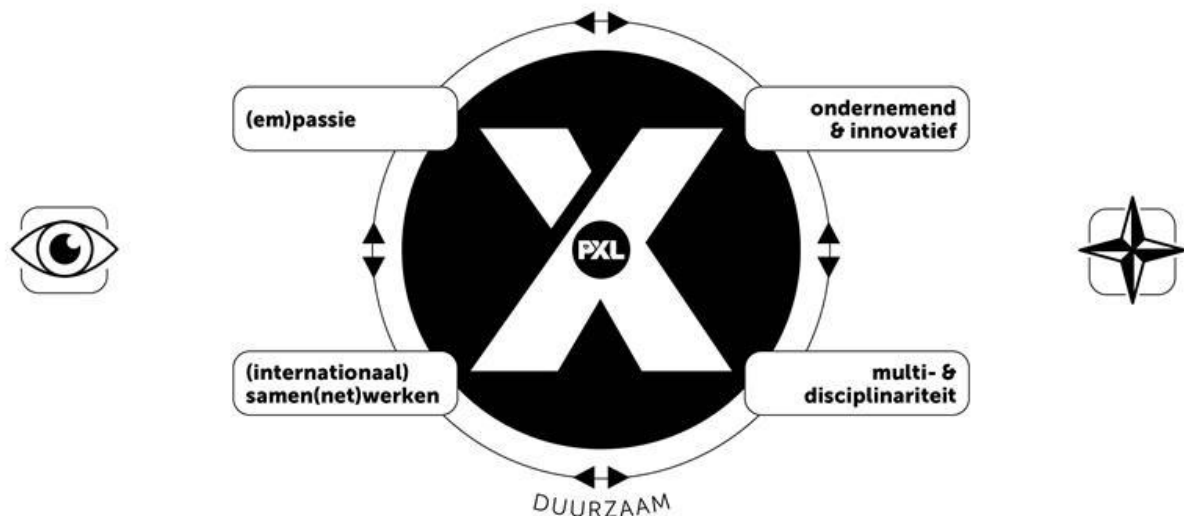


Figure 45: X-Factor guide

5 Conclusion

This research focused on the development of a modular unmanned aerial vehicle capable of performing object avoidance using artificial intelligence, while remaining suitable for operation in harsh environments and adaptable for future applications. The problem context highlighted the need for such a platform within both research and practical domains, particularly for use cases such as search and rescue operations, communication support, and as a learning platform for future development. To address this objective, a structured methodology was followed in which component selection, system design, and subsystem validation were carried out iteratively. Key design decisions were based on weather performance and utilizing tools to determine suitable parts for the drone. The UAV platform was constructed using a scalable frame design, appropriate propulsion components, and a flight controller capable of stable and reliable operation. In parallel, an AI subsystem based on a companion computer with a depth camera was developed to enable perception and decision-making for obstacle avoidance.

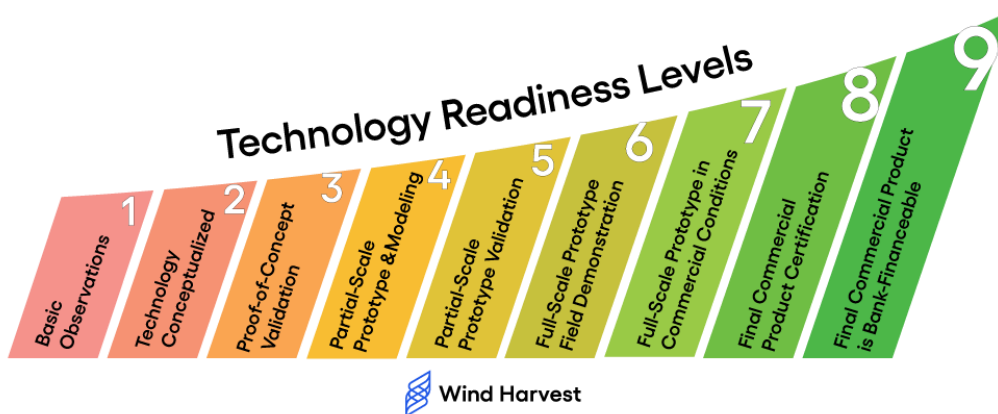


Figure 46: TRL guide

The results demonstrate that the UAV platform is successfully realized at a Technology Readiness Level of approximately 2–3. The drone system is fully assembled and operational in manual control. In addition, the AI subsystem can perform object detection and basic avoidance logic in isolation. These results confirm that both the hardware platform and the perception pipeline are technically feasible and function as intended within their respective domains. However, the research also shows that full system integration remains the main challenge. While individual subsystems perform correctly, the communication between the companion computer and the flight controller is not yet sufficiently reliable to enable autonomous flight. This leads to the conclusion that the core concept of an AI-driven object-avoiding UAV is feasible, but that achieving complete system integration requires additional development time, testing, and refinement. This project contributes to the advancement of modular UAV systems that with further development be adapted for multiple applications in both research and industry. The developed platform provides a solid foundation for future work, not only within the context of this project but also for other use cases such as environmental monitoring, communication systems, and educational purposes. Looking forward, future research should focus on improving system integration, particularly the communication between perception and control layers. Additionally, further validation through real-world testing and the potential integration of additional sensors can enhance system robustness and expand functionality. By building upon the established foundation, the platform can evolve toward a fully autonomous UAV system capable of operating reliably in complex and demanding environments.

6 Bibliografy

- Ardupilot. (n.d.). Retrieved from <https://ardupilot.org/copter/docs/common-rangefinder-landingpage.html>
- droneinfo. (2023, 11 22). *droneinfo*. Retrieved from droneinfo.fi: <https://droneinfo.fi/en/news/changes-flying-drone-starting-1-january-2024>
- Hobbyking. (n.d.). *Hobbyking.com*. Retrieved from Hobbyking: https://hobbyking.com/de_de/blog/drone-battery-temperature-guide-safe-lipo-temps/?srsId=AfmBOoqlOt3fufQUJZG1sQ-token0M_0wfgir-UFvJubNxyS3B3wOHkDk&__store=de_de
- IEEE. (2016, 12 31). *IEEE Xplore*. Retrieved from [ieeexplore.ieee.org: https://ieeexplore.ieee.org/document/7747236](https://ieeexplore.ieee.org/document/7747236)
- Intel. (n.d.). *mouser*. Retrieved from https://www.mouser.com/pdfdocs/Intel_D400_Series_Datasheet.pdf?srsId=AfmBOoq5IERIfaGZbnRHxzma5LDD7m4PV39YTNDwX5n-OkcnpAazfbTu
- Jutai, J. W. (2023, 09 29). *mdpi.com*. Retrieved from mdpi: <https://www.mdpi.com/2227-7080/11/6/168>
- Lapin AMK. (2025, 12 31). *frostbit*. Retrieved from frostbit: <https://www.frostbit.fi/en/portfolio/agrarsense-5g/>
- Lapin AMK. (n.d.). *frostbit*. Retrieved from frostbit.fi: <https://www.frostbit.fi/en/about-us/>
- Müller, M. (n.d.). *ecal*. Retrieved from [ecal](https://www.ecalc.com/).
- PXL. (n.d.). *pxl.be*. Retrieved from PXL: <https://www.pxl.be/over-pxl/>
- Roe, G. (2023, 12 18). *lidarnews*. Retrieved from lidarnews.com: <https://lidarnews.com/lidar-cameras-and-radar-compared/>
- Terabee. (n.d.). *youtube.com*. Retrieved from youtube: <https://www.youtube.com/watch?v=ig8IRTejn8A>
- Vlaanderen. (n.d.). *vlaanderen.be*. Retrieved from <https://www.vlaanderen.be/uw-overheid/beleid/het-vlaamse-beleid-voor-duurzame-ontwikkeling/sdgs-in-vlaanderen>

https://docs.px4.io/main/en/getting_started/rc_transmitter_receiver

https://docs.px4.io/main/en/getting_started/rc_transmitter_receiver#compatible_receivers

https://originhobbies.com/shop/radio-control/radio-receivers/radiomaster-rp3-h-expresslrs-elrs-2-4ghz-nano-receiver/?srsltid=Afm-BOorwU9A_ILxDHf4Ent9CbzpZq8yj8xsoCWCibmKw_G6KG2E4VGmV&v=75778bf8fde7

https://docs.px4.io/main/en/getting_started/rc_transmitter_receiver

https://docs.px4.io/main/en/getting_started/rc_transmitter_receiver#compatible_receivers

https://docs.px4.io/main/en/telemetry/crsf_telemetry#expresslrs-radio-systems

https://docs.px4.io/main/en/frames_multicopter/dji_f450_cuav_5plus

https://docs.px4.io/main/en/telemetry/holybro_sik_radio

<https://holybro.com/collections/telemetry-radios/products/sik-telemetry-radio-v3>

<https://youtu.be/jclh4U3vw3A>

<https://www.calculatorsoup.com/calculators/physics/kinetic.php>

https://developer.download.nvidia.com/assets/embedded/secure/jetson/orin_nano/docs/Jetson-Orin-Nano-DevKit-Carrier-Board-Specification_SP-11324-001_v1.3.pdf?to-ken=exp=1771845507~hmac=793c1f44ac75df8a357773666619c3b559f1f408e753a0e6b3975322dedfc17c&t=eyJscyl6ImdzZW8iLCJsc2QiOiJodHRwciovL3d3dy5nb29nbGUuY29tLyJ9

<https://github.com/DFRobotdl/DFROBOTDL/blob/main/DFR1122/DFR1122%20Datasheet.pdf>

<https://oscarliang.com/propellers/>

<https://goldenkoltsova.blogspot.com/2019/10/holybro-s500-v2-pixhawk-4-480mm.html>

<https://docs.px4.io/main/en/ros2/>

https://www.vertiq.co/40-06_g2?utm_source=unmannedsystemstechnology.com&utm_medium=referral

<https://youtu.be/ketmb8u2UI>

<https://youtu.be/uC9hVygGvDE?si=msy2YrCmQLr-eMPO>

<https://youtu.be/QL62iRkLX3s?si=pi0iXism9DTkg0jo>

https://valuehobby.com/3536-920kv-outrunner.html?fbclid=IwY2xjawP7pplleH-RuA2FlbQlxMABicmlkETBSN3BvSEtGcEs4a0hNRmMxc3JOYwZhchBfaWQQMjlyMDM5MTc4ODI-wMDg5MgABHmwdKiVMQH-BL5W6zSIKde76bg7HjHSVb5lfNuZ8cCdRJwabH_0IOP62GzY_aem_0coc7C47DXDwRg8cpXzyHg

<https://www.aos-rc.com/fpv-gear/radio-transmitters>

https://valuehobby.com/3536-920kv-outrunner.html?fbclid=IwY2xjawP7pplleH-RuA2FlbQlxMABicmlkETBSN3BvSEtGcEs4a0hNRmMxc3JOYwZhchBfaWQQMjlyMDM5MTc4ODI-wMDg5MgABHmwdKiVMQH-BL5W6zSIKde76bg7HjHSVb5lfNuZ8cCdRJwabH_0IOP62GzY_aem_0coc7C47DXDwRg8cpXzyHg

https://www.vertiq.co/40-06-g2?utm_source=unmannedsystemstechnology.com&utm_medium=referral

https://iflight-rc.eu/en-fi/products/xing2-2806-5-fpv-cinelifiter-motor?variant=55991470096649&utm_medium=product_sync&utm_source=google&utm_content=sag_organic&utm_campaign=sag_organic&gad_source=1&gad_campaignid=23041099353&gbraid=0AAAAAqCN977Dp7XMZXTeHht2GQEwax-wJF&gclid=CjwKCAiAtLvMBhB_Eiwa1u6_Pm1ac6AuULHjGdGi-hoE42E7NU4hPKuXch6dXj1l1j8X1V16GrBdZ-xoCdjEQAvD_BwE

<https://youtu.be/61duFM-LXhI?si=BwDmEk7suafZjast>

<https://www.youtube.com/watch?v=tIE8IN71UFI&list=PLTHi8SkNXszk3vEXVvE4HUq0Az-MxIO5C&index=2>

<https://pdf.aeroexpo.online/pdf/videodrone/geodrone-x4l/181486-18213.html>

<https://oscarliang.com/flight-controller/>

https://docs.px4.io/main/en/assembly/quick_start_pixhawk6c

<https://oscarliang.com/esc/>

https://docs.px4.io/main/en/assembly/quick_start_pixhawk6c

<https://www.thingiverse.com/thing:7143191>

<https://oscarliang.com/fly-fpv-winter/>

<https://www.ecalc.ch/xcoptercalc.php>

<https://droneinfo.fi/en/study-material/airspace-restrictions-are-you-aware-rules>

<https://www.skydroid.xin/#/serviceSupport/downloadCenter>

https://toyscentral.nz/products/small-electric-motor-brushless-motor-kv850-dc-motor-multiaxis-selflocking-brushless-motor-cwccw-electrical-accessories-for-diy-airplane-model-speedrpm-cw?srsId=Afm-BOOrAXyNQY04_7wCPHW8M4pgr9Wp4YW-Gh9AciTTe1SJ9aaz6Q0bq&utm_source=chatgpt.com

<https://fi.rsdelivers.com/product/okdo/54010012/okdo-lidar-module-with-bracket-development-kit-for/2037609>

<https://www.youtube.com/watch?v=aiFxVJFjlos>

<https://docs.arkelectron.com/flight-controller/ark-jetson-pab-carrier/block-diagram>

<https://holybro.com/products/pixhawk-6c?srsId=AfmBOOrkQeeuvek92LnENsyYe7gM3-w0c0pjEa6XZD0-EF9t17OJhCCR>

<https://realsenseai.com/wp-content/uploads/2024/10/Intel-RealSense-D400-Series-Datasheet-October-2024.pdf>

<https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin>

<https://www.miniquadtestbench.com/escs/>

https://www.ecalc.ch/calcinclue/help/xcoptercalchelp.htm#flat_vs_coax

<https://www.stereolabs.com/en-fi/products/zed-2>

https://docs.qgroundcontrol.com/master/en/qgc-user-guide/plan_view/plan_view.html

<https://www.stereolabs.com/en-fi/blog/getting-started-with-ros2-on-jetson-agx-orin>

<https://docs.px4.io/main/en/config/airframe>

Code prototype 1 flight controller communication:

```
from pymavlink import mavutil

import time

SERIAL_PORT = '/dev/ttyTHS1'
BAUD_RATE = 57600

class DroneController:
    """A comprehensive class to control a drone via MAVLink."""
    def __init__(self, port, baud):
        print(f"Connecting to Pixhawk on {port} at {baud} baud...")
        self.master = mavutil.mavlink_connection(port, baud=baud)
        self.wait_for_heartbeat()

    def wait_for_heartbeat(self):
        print("Waiting for heartbeat...")
        self.master.wait_heartbeat()
        print(f"Heartbeat from system (system {self.master.target_system} component {self.master.target_component})")

    def set_mode(self, mode):
        print(f"Setting mode to {mode}...")
        # Check if mode is available
        if mode not in self.master.mode_mapping():
            print(f"Unknown mode : {mode}")
            print(f"Available modes:", list(self.master.mode_mapping().keys()))
            return

        mode_id = self.master.mode_mapping()[mode]
        self.master.set_mode(mode_id)

        # Wait for ACK
        ack = self.master.recv_match(type='COMMAND_ACK', blocking=True, timeout=3)
        print(f"Set Mode ACK: {ack}")

    def arm(self):
        print("Arming motors...")
        self.master.mav.command_long_send(
            self.master.target_system, self.master.target_component,
            mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM, 0,
            1, 21196, 0, 0, 0, 0, 0
        )
        self.master.motors_armed_wait()
        print("Armed!")

    def disarm(self):
        print("Disarming motors...")
        self.master.mav.command_long_send(
            self.master.target_system, self.master.target_component,
```

```

        mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM, 0,
        0, 21196, 0, 0, 0, 0, 0
    )
    self.master.motors_disarmed_wait()
    print("Disarmed!")

def takeoff(self, altitude):
    print(f"Taking off to {altitude}m...")
    self.master.mav.command_long_send(
        self.master.target_system, self.master.target_component,
        mavutil.mavlink.MAV_CMD_NAV_TAKEOFF, 0,
        0, 0, 0, 0, 0, altitude
    )
    ack = self.master.recv_match(type='COMMAND_ACK', blocking=True, timeout=3)
    print(f"Takeoff ACK: {ack}")

def set_velocity_body(self, vx, vy, vz, yaw_rate=0.0):
    # Sends velocity commands in GUIDED mode (m/s and rad/s)
    self.master.mav.set_position_target_local_ned_send(
        0, self.master.target_system, self.master.target_component,
        mavutil.mavlink.MAV_FRAME_BODY_NED,
        int(0b011111000111), # type_mask (only speeds enabled)
        0, 0, 0, # x, y, z positions (not used)
        vx, vy, vz, # x, y, z velocity in m/s
        0, 0, 0, # x, y, z acceleration (not used)
        0, yaw_rate # yaw, yaw_rate
    )

def send_rc_override(self, channels):
    # channels is a list of 8 PWM values (1000-2000), 0 means release
    # e.g., rc = [1500, 1500, 1500, 1500, 0, 0, 0, 0] # Roll, Pitch, Throttle,
Yaw
    self.master.mav.rc_channels_override_send(
        self.master.target_system, self.master.target_component,
        *channels
    )

def request_message_interval(self, message_id, frequency_hz):
    self.master.mav.command_long_send(
        self.master.target_system, self.master.target_component,
        mavutil.mavlink.MAV_CMD_SET_MESSAGE_INTERVAL, 0,
        message_id,
        1e6 / frequency_hz,
        0, 0, 0, 0, 0
    )

if __name__ == "__main__":
    drone = DroneController(SERIAL_PORT, BAUD_RATE)

    print("Listening... Waiting for the drone to be ARMED and in ALT_HOLD mode.")

```

```

print("You can arm via RC or Ground Control Station.")

# Request RAW_IMU (message ID 27) at 10Hz to ensure we get gyro data
drone.request_message_interval(mavutil.mavlink.MAVLINK_MSG_ID_RAW_IMU, 10)

try:
    while True:
        # We must process ALL pending messages to keep internal flightmode and
        # armed states perfectly up-to-date
        msg = drone.master.recv_match(blocking=True, timeout=0.05)
        while msg:
            # Exclude BAD_DATA to avoid cluttering if you only want valid
            # mavlink messages
            if msg.get_type() != 'BAD_DATA':
                pass # print(msg) - You can uncomment this to print everything,
                # but it will be very fast!
                if msg.get_type() == 'RAW_IMU':
                    print(f"Gyro Data: X:{msg.xgyro} Y:{msg.ygyro}
                    Z:{msg.zgyro}")
                elif msg.get_type() == 'HEARTBEAT':
                    print(f"Heartbeat - Mode: {drone.master.flightmode}, Armed:
                    {drone.master.motors_armed()}")

            msg = drone.master.recv_match(blocking=False)

        is_armed = drone.master.motors_armed()
        current_mode = drone.master.flightmode

        if is_armed and current_mode == "ALT_HOLD":
            print("==> Detected ARMED + ALT_HOLD context! Initiating motor spin
            sequence...")

            # Spin up throttle from 1000 to 2000
            for pwm in range(1000, 2001, 50):
                # Periodically check if the user disarmed or switched modes
                # mid-sequence
                msg = drone.master.recv_match(blocking=False)
                while msg:
                    if msg.get_type() != 'BAD_DATA':
                        if msg.get_type() == 'RAW_IMU':
                            print(f"Gyro Data: X:{msg.xgyro} Y:{msg.ygyro}
                            Z:{msg.zgyro}")
                        elif msg.get_type() == 'HEARTBEAT':
                            print(f"Heartbeat - Mode: {drone.master.flight-
                            mode}, Armed: {drone.master.motors_armed()}")
                    msg = drone.master.recv_match(blocking=False)

                if not drone.master.motors_armed() or drone.master.flightmode
                != "ALT_HOLD":
                    print("State changed mid-spin. Aborting sequence.")

```

```

        break

        drone.send_rc_override([1500, 1500, pwm, 1500, 0, 0, 0, 0])
        time.sleep(0.1)

    # Spin down throttle from 2000 to 1000
    for pwm in range(2000, 999, -50):
        msg = drone.master.recv_match(blocking=False)
        while msg:
            if msg.get_type() != 'BAD_DATA':
                if msg.get_type() == 'RAW_IMU':
                    print(f"Gyro Data: X:{msg.xgyro} Y:{msg.ygyro}
Z:{msg.zgyro}")

                    elif msg.get_type() == 'HEARTBEAT':
                        print(f"Heartbeat - Mode: {drone.master.flight-
mode}, Armed: {drone.master.motors_armed()}")
                        msg = drone.master.recv_match(blocking=False)

            if not drone.master.motors_armed() or drone.master.flightmode
!= "ALT_HOLD":
                print("State changed mid-spin. Aborting sequence.")
                break

        drone.send_rc_override([1500, 1500, pwm, 1500, 0, 0, 0, 0])
        time.sleep(0.1)

    else:
        # Neutralize overrides if we are not in the active test state
        drone.send_rc_override([0, 0, 0, 0, 0, 0, 0, 0])

except KeyboardInterrupt:
    print("\nInterrupted by user. Exiting...")
finally:
    print("Releasing RC Override and returning control...")
    drone.send_rc_override([0, 0, 0, 0, 0, 0, 0, 0])

```

Code prototype 2 depth camera:

```
import pyrealsense2 as rs

import numpy as np
import cv2
import os

## https://github.com/jetsonhacks/jetson-orin-librealsense camera library for jet-
son devices. It is a wrapper around the Intel Realsense SDK.
## The code is adapted from the librealsense examples and modified to include spa-
tial filtering, hole filling, and a simple obstacle detection algorithm based on
dividing the depth image into 6 segments and checking
## testing code https://github.com/carlosefrias/realsense-t265-and-d435-test-
ing/blob/master/Python/realsenseD435_stream.py

def image_file_counter(path):
    files = 0
    for _, _, filenames in os.walk(path):
        files += len(filenames)
    return files + 1

def spatial_filtering(depth_frame, magnitude=2, alpha=0.5, delta=20, holes_fill=0):
    spatial = rs.spatial_filter()
    spatial.set_option(rs.option.filter_magnitude, magnitude)
    spatial.set_option(rs.option.filter_smooth_alpha, alpha)
    spatial.set_option(rs.option.filter_smooth_delta, delta)
    spatial.set_option(rs.option.holes_fill, holes_fill)
    depth_frame = spatial.process(depth_frame)
    return depth_frame

def hole_filling(depth_frame):
    hole_filling = rs.hole_filling_filter()
    depth_frame = hole_filling.process(depth_frame)
    return depth_frame

def check_blocked(depth_roi, depth_scale, threshold_m=1.0):
    """
    Check if a region of interest (ROI) is blocked.
    Calculates the median depth of valid pixels and compares it to the threshold.
    """
    dist_m = depth_roi * depth_scale
    valid_dist = dist_m[dist_m > 0]

    if len(valid_dist) < (depth_roi.size * 0.1):
        # Too little valid data, assume blocked/too close for safety
        return True

    median_dist = np.median(valid_dist)
    return median_dist < threshold_m
```

```
# define global variables
# =====
# file names and paths
rgb_img_path = 'captured_images/rgb_image/'
depth_img_path = 'captured_images/depth_image/'
colored_depth_img_path = 'captured_images/coloured_depth_image/'
intrinsics = True
rotate_camera = False

if __name__ == "__main__":
    # =====
    # 1. Configure all streams
    # =====
    pipeline = rs.pipeline()
    config = rs.config()
    config.enable_stream(rs.stream.depth, 640, 480, rs.format.z16, 30)
    config.enable_stream(rs.stream.color, 640, 480, rs.format.bgr8, 30)

    # =====
    # 2. Start the streaming
    # =====
    print("Starting up the Intel Realsense D435...")
    print("")
    profile = pipeline.start(config)

    # =====
    # 3. The depth sensor's depth scale
    # =====
    depth_sensor = profile.get_device().first_depth_sensor()
    depth_scale = depth_sensor.get_depth_scale()
    print("Depth Scale is: ", depth_scale)
    print("")

    # =====
    # 4. Create an align object.
    #     Align the depth image to the rgb image.
    # =====
    align_to = rs.stream.color
    align = rs.align(align_to)

    try:
        # =====
        # 5. Skip the first 30 frames.
        # This gives the Auto-Exposure time to adjust
        # =====
        for x in range(30):
            frames = pipeline.wait_for_frames()
            # Align the depth frame to color frame
```

```

        aligned_frames = align.process(frames)

print("Intel Realsense D435 started successfully.")
print("")

while True:
    # =====
    # 6. Wait for a coherent pair of frames:
    # =====
    frames = pipeline.wait_for_frames()

    # =====
    # 7. Align the depth frame to color frame
    # =====
    aligned_frames = align.process(frames)

    # =====
    # 8. Fetch the depth and colour frames from stream
    # =====
    depth_frame = aligned_frames.get_depth_frame()
    color_frame = aligned_frames.get_color_frame()
    if not depth_frame or not color_frame:
        continue

    # print the camera intrinsics just once. it is always the same
    if intrinsics:
        print("Intel Realsense D435 Camera Intrinsics: ")
        print("=====")
        print(depth_frame.profile.as_video_stream_profile().intrinsics)
        print(color_frame.profile.as_video_stream_profile().intrinsics)
        print("")
        intrinsics = False

    # =====
    # 9. Apply filtering to the depth image
    # =====
    # Apply a spatial filter without hole_filling (i.e. holes_fill=0)
    depth_frame = spatial_filtering(depth_frame, magnitude=2, alpha=0.5,
delta=50, holes_fill=0)
    # Apply hole filling filter
    depth_frame = hole_filling(depth_frame)

    # =====
    # 10. colourise the depth map
    # =====
    depth_color_frame = rs.colorizer().colorize(depth_frame)

    # =====
    # 11. Convert images to numpy arrays
    # =====

```

```

depth_image = np.array(depth_frame.get_data())
depth_color_image = np.asanyarray(depth_color_frame.get_data())
color_image = np.asanyarray(color_frame.get_data())

#
=====
# 12. Only rotate the images if the realsense camera is placed verti-
cal.
# Otherwise set the variable "rotate_camera = False"
#
=====
if rotate_camera:
    depth_image = np.rot90(depth_image)
    depth_color_image = np.rot90(depth_color_image)
    color_image = np.rot90(color_image)

# =====
# 13. Create Dynamic Grid and Calculate Obstacles
# =====
h, w = depth_image.shape
ROWS, COLS = 4, 5
w_step = w // COLS
h_step = h // ROWS

# Create a 2D list to store block statuses
blocked_grid = [[False for _ in range(COLS)] for _ in range(ROWS)]

# Check if each segment is blocked (Threshold is 1.0 meters)
for r in range(ROWS):
    for c in range(COLS):
        y_start, y_end = r * h_step, (r + 1) * h_step if r < ROWS - 1
        else h
        x_start, x_end = c * w_step, (c + 1) * w_step if c < COLS - 1
        else w

        segment = depth_image[y_start:y_end, x_start:x_end]
        blocked_grid[r][c] = check_blocked(segment, depth_scale, 1.0)

# Determine Action based on rules
# We adapt previous rules to the new grid scale:
total_blocks = sum([sum(row) for row in blocked_grid])
bottom_row_blocked = sum(blocked_grid[-1]) >= (COLS - 1) # Most of
bottom row is blocked

# Check left columns (e.g. columns 0 and 1)
left_blocks = sum([blocked_grid[r][c] for r in range(ROWS) for c in
range(2)])

# Check right columns (e.g. columns COLS-2 and COLS-1)
right_blocks = sum([blocked_grid[r][c] for r in range(ROWS) for c in
range(COLS-2, COLS)])

```

```

        action = "HOVER"

        if total_blocks >= (ROWS * COLS) - 3: # Almost all strictly blocked
            action = "MOVE BACK"
        elif bottom_row_blocked:
            action = "MOVE UP"
        elif left_blocks > right_blocks and left_blocks >= ROWS:
            action = "MOVE RIGHT"
        elif right_blocks > left_blocks and right_blocks >= ROWS:
            action = "MOVE LEFT"

        print(f"Decision: {action} | Blocks: {total_blocks}/{ROWS*COLS} | Left:
{left_blocks} Right: {right_blocks} Bot: {sum(blocked_grid[-1])}")

        # Draw lines and text for regions in both color and depth_color
        for img in (color_image, depth_color_image):
            # Vertical lines
            for c in range(1, COLS):
                cv2.line(img, (c * w_step, 0), (c * w_step, h), (255, 255,
255), 1)

            # Horizontal lines
            for r in range(1, ROWS):
                cv2.line(img, (0, r * h_step), (w, r * h_step), (255, 255,
255), 1)

        def put_status(img, blocked, pt):
            text = "X" if blocked else "O"
            color = (0, 0, 255) if blocked else (0, 255, 0)
            # Small text for tighter grids
            cv2.putText(img, text, pt, cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)

        for r in range(ROWS):
            for c in range(COLS):
                put_status(color_image, blocked_grid[r][c], (c * w_step + 10, r
* h_step + 25))

        cv2.putText(color_image, f"ACTION: {action}", (20, h - 20),
cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0), 3)

        # Stack rgb and depth map images horizontally for visualisation only
        images = np.hstack((color_image, depth_color_image))

        # Show horizontally stacked rgb and depth map images
        cv2.namedWindow('RGB and Depth Map Images')
        cv2.imshow('RGB and Depth Map Images', images)
        c = cv2.waitKey(1)

        # =====
        # If the 's' key is pressed, we save the images

```

```
# =====
if c == ord('s'):
    img_counter = image_file_counter(rgb_img_path)

    '''create a stream folders'''
    if not os.path.exists(rgb_img_path):
        os.makedirs(rgb_img_path)
    if not os.path.exists(depth_img_path):
        os.makedirs(depth_img_path)
    if not os.path.exists(colored_depth_img_path):
        os.makedirs(colored_depth_img_path)

    filename = str(img_counter) + '.png'
    filename_raw = str(img_counter) + '.raw'
    # save the rgb colour image
    cv2.imwrite(os.path.join(rgb_img_path, filename), color_image)
    # Save the depth image in raw binary format uint16.
    f = open(os.path.join(depth_img_path, filename_raw), mode='wb')
    depth_image.tofile(f)
    cv2.imwrite(os.path.join(colored_depth_img_path, filename),
depth_color_image)

    print('images have been successfully saved')

elif c == 27: # esc to exit
    break

finally:
    # Stop streaming
    pipeline.stop()
```